ECC D0701

RIZVI EDUCATION SOCIETY

Rizvi College of Engineering

Department of Electronics and Computer Science Engineering

Deep Learning Lab Journal



Name:

Class:

Roll no. / UIN:

RIZVI EDUCATION SOCIETY'S

Rizvi College of Engineering

Department of Electronics and Computer Science Engineering

Lab File

Class & Semester: *BE (Sem 7)*

Subject: *Deep Learning Lab*

Name: ……

Roll No.: ……..

UIN: ……

Document Revised on: *04/ 10 / 2024*

RIZVI EDUCATION SOCIETY'S

Rizvi College of Engineering

Department of Electronics & Computer Science

Certificate

This is to certify that.....of Fourth year Electronics & Computer Science Engineering Department has performed all the experiments of Deep Learning with satisfactory results.

……..

Subject In-charge

INSTITUTE VISION

To become a leading entity in transforming the diverse class of learners into innovators, analyzers and entrepreneurs competent to develop eco-friendly sustainable solutions and work in a multi-disciplinary environment to meet the global challenges and contribute towards nation building.

INSTITUTE MISSION

IM1. Impart Core Fundamental Principles: To impart core fundamental principles of engineering and science that will enable the learner to develop solutions to complex engineering problems, through conventional and innovative teaching learning methods and mentoring.

IM2. Bridge the Technical Skill Gap: To bridge the technical skill gap through curriculum enrichment activities for industry readiness.

IM3. Inculcate Professional Etiquettes and Ethics: To groom the learner through dedicated training, placement and extension activities and to inculcate professional etiquettes and ethics aimed at holistic development of the learners enabling them to acquire distinguished positions in the leading industries or be eligible for higher studies in globally recognized universities.

IM4. Research and Development: To provide modern infrastructure and the necessary resources, for planning and implementing innovative ideas, leading to meaningful research and development and entrepreneurship.

PROGRAM OUTCOMES

PO1. Engineering knowledge
PO2. Problem analysis
PO3. Design/development of solutions
PO4. Conduct investigations of complex problems
PO5. Modern tool usage
PO6. The engineer and society
PO7. Environment and sustainability
PO8. Ethics
PO9. Individual and team work
PO10. Communication
PO11. Project management and finance
PO12. Life-long learning

Course Objectives

- To implement basic neural network models.
- To implement various training algorithms for feedforward neural networks.
- To design deep learning models for supervised, unsupervised and sequence learning.

Course Outcomes

Learners will be able to…

CO1. Implement basic neural network models.
CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.
CO3. Build and train deep learning models such as Autoencoders, CNNs, RNN, LSTM, GRU etc.

Rubrics

Following rubrics will be used to assess the work submitted by the students.

1. For Experiment 1 to 10

Criteria 1: Knowledge				
Knowledge about the concept				
5	**4**	**3**	**2**	**1**
Excellent	Very Good	Good	Average	Poor
Topics very clear	Topics need brush-up	Good explanation of topics	Average explanation of topics	Poor explanation of topics
Criteria 2: Delivery				
Submission in specified duration				
5	**4**	**3**	**2**	**1**
Excellent	Very Good	Good	Average	Poor
Submitted before the due date	Submitted on the due date	Submitted within one week past the due date	Submitted within two weeks past the due date	Submitted after weeks past the due date
Criteria 3: Outputs & Conclusion				
Completion of all steps and concluded properly				
5	**4**	**3**	**2**	**1**
Excellent	Very Good	Good	Average	Poor
Output attached without any discrepancy and concluded well uniquely.	Output proper and concluded well.	Output proper with extremely low plagiarism.	Output attached is not clear with some amount of plagiarism.	Incorrect output with complete plagiarism. Not accepted.

1. **For Assignments**

Criteria 1 : In time Submission			
Submission of the experiment in specified duration			
5	**4**	**3**	**1**
Excellent	Good	Average	Poor
Submitted on or before time	Submitted within one week past due date	Submitted within two weeks past due date	Submitted after two weeks past due date
Criteria 2 : Presentation	**How well the content is presented**		
5	**4**	**3**	**1**
Excellent	Good	Average	Poor
Content presentation with proper figures, tables etc.	Content Presentation with few missing figures, tables etc.	Content presentation is missing figures etc	Content presentation is not at all proper.

Lab Guidelines

- Do not go on unapproved sites.
- No food or drinks near the keyboard.
- Only use your assigned computer and workstation.
- Do not change the settings on the computer.
- Ask permission to download.
- Ask permission to print documents
- Save your work often.
- Report cyberbullying.

INDEX

Sr. No.	Title	Page No.	Performed On	Sign & Date	Rubrics Points
1	Exp 1: To Explore Python libraries for deep learning example) Tensor flow, Keras etc.	11	24/7/2025		
2	Exp 2: Implement McCulloch Pitts model for binary logic function.	19	7/8/2025		
3	Exp 3: Implement Perceptron algorithm to simulate any logic gate.	26	25/8/2025		
4	Exp 4: Implement Multilayer Perceptron algorithm to simulate XOR gate.	32	25/8/2025		
5	Exp 5: Apply Stochastic Gradient Descent on a Supervised Single layer feed forward neural network.	40	1/9/2025		
6	Exp 6: Apply Mini Batch GD on a supervised single layer feed forward neural network.	47	1/9/2025		
7	Exp 7: Implement a backpropagation algorithm to train a DNN with at least two hidden layers.	54	15/9/2025		
8	Exp 8: Design and implement a CNN model for digit recognition application.	60	15/9/2025		
9	Exp 9: Design the architecture & implement auto-encoder model for image compression.	65	22/9/2025		
10	Exp 10: Design and implement RNN for text data.	70	22/9/2025		
11	Assignment 1	76			
12	Assignment 2				
				TOTAL	

EXPERIMENTS

Expt. No. 1

Title: To explore python libraries for deep learning examples: Keras, TensorFlow.

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 24/7/2025

Sign:_____

Experiment No. 1

Deep Learning

Aim : To explore python libraries for deep learning eg) Keras, TensorFlow.

Software : Google Collab

Pre-Lab Questions:

1. Explain the features of Tensor flow library?
   TensorFlow is an open-source deep learning library for scalable model training with multi-device support and easy APIs.

Post-Lab Questions:

1. Explain the applications that can be implemented using Tensor Flow and Keras ?
   TensorFlow and Keras can be used to build applications like image recognition, natural language processing, and predictive analytics.

Theory : TensorFlow is an open-source framework developed by Google researchers to run machine learning, deep learning and other statistical and predictive analytics workloads. Like similar platforms, designed to streamline the process of developing and executing advanced analytics applications for users such as data scientists, statisticians and predictive modelers.

Features:

1. Prepare and load data for successful ML outcomes:

Data can be the most important factor in the success of your ML endeavors. TensorFlow offers multiple data tools to help you consolidate, clean and pre-process data at scale:

- Standard datasets for initial training and validation
- Highly scalable data pipelines for loading data
- Pre-processing layers for common input transformations
- Tools to validate and transform large datasets
- Additionally, responsible AI tools help you uncover and eliminate bias in your data to produce fair, ethical outcomes from your models.

2. Build and fine-tune models with the TensorFlow ecosystem:

TensorFlow supports distributed training, immediate model iteration and easy debugging with Keras , and much more. Tools like Model Analysis and Tensor Board help you track development and improvement through your model's lifecycle.

3. Deploy models on-device, in the browser, on-prem, or in the cloud:

TensorFlow provides robust capabilities to deployyour models on any environment - servers, edgedevices, browsers, mobile, microcontrollers, CPUs, GPUs, FPGAs. TensorFlow Serving can run ML models a

production scale on the most advanced processors in the world, including Google's custom TensorProcessing Units (TPUs). If you need to analyze data close to its source to reduce latency and improve data privacy, the TensorFlow Lite framework lets you run models on mobile devices, edge computing devices, and even microcontrollers, and the TensorFlow.js framework lets you run machine learning with just a web browser.

4. Implement MLOps for production ML: The TensorFlow platform helps you implement best practices for data automation, model tracking, performance monitoring, and model retraining. Using production-level tools to automate and track model training over the lifetime of a product, service, or business process is critical to success. TFX provides software frameworks and tooling for full MLOps deployments, detecting issues as your data and models evolve over time.

Keras

Definition: Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. It also has extensive documentation and developer guides.

Features:

1. Used for experimenting: Keras is the most used deep learning framework among top-5 winning teams on Kaggle. Because Keras makes it easier to run new experiments, it empowers you to try more ideas than your competition, faster.
2. Exascale machine learning: Built on top of TensorFlow 2, Keras is an industry-strength framework that can scale to large clusters of GPUs or an entire TPU pod. It's not only possible; is easy.
3. Deployment: Take advantage of the full deployment capabilities of the TensorFlow platform. You can export Keras models to JavaScript to run directly in the browser, to TF Lite to run on iOS, Android, and embedded devices. it is also easy to serve Keras models as via a web API.
4. Vast ecosystem:

Keras is a central part of the tightly-connected TensorFlow 2 ecosystem, covering every step of the machine learning workflow, from data management to hyperparameter training to deployment solutions.

5. State-of-the-art research:

Keras is used by CERN, NASA, NIH, and many more scientific organizations around the world (and yes, Keras is used at the LHC). Keras has the low-level flexibility to implement arbitrary research ideas while offering optional high-level convenience features to speed up experimentation cycles.

Programs & Output:

Tensorflow:

```
# importing the dependencies
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import numpy as np
import matplotlib.pyplot as plt

# Model Parameters
learning_rate = 0.01 # determines the step size at each iteration while moving toward a minimum of a loss function.
training_epochs = 2000 # An epoch is when all the training data is used at once and is defined as the total number of iteration
display_step = 200

# Training Data
train_X =
np.asarray([3.3,4.4,5.5,6.71,6.93,4.168,9.779,6.182,7.59,2.167,7.042,10.791,5.313,7.997,5.654,9.27,3.1])
train_y =
np.asarray([1.7,2.76,2.09,3.19,1.694,1.573,3.366,2.596,2.53,1.221,2.827,3.465,1.65,2.904,2.42,2.94,1.3])
n_samples = train_X.shape[0] # assigns the number of samples in the train_X dataset to the variable n_samples.

# Test Data
test_X = np.asarray([6.83, 4.668, 8.9, 7.91, 5.7, 8.7, 3.1, 2.1]) # in NumPy it is used to convert a given input (such as a list, tuple, or array-like object) into an array.
test_y = np.asarray([1.84, 2.273, 3.2, 2.831, 2.92, 3.24, 1.35, 1.03])
# Set placeholders for feature and target vectors
X = tf.placeholder(tf.float32) # sed to define a placeholder tensor that could be later fed with actual data during the execution of a computational graph.
y = tf.placeholder(tf.float32) # float32, indicates that it will hold floating-point values.

# Set model weights and bias
W = tf.Variable(np.random.randn(), name="weight")
b = tf.Variable(np.random.randn(), name="bias")

# Construct a linear model
linear_model = W*X + b
# Mean squared error
cost = tf.reduce_sum(tf.square(linear_model - y)) / (2*n_samples) # represents the mean squared error (MSE) cost function for a linear regression model.

# Gradient descent
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost) # TensorFlow sets up the computation to minimize the cost function using gradient descent.

# Initializing the variables
init = tf.global_variables_initializer() # get an operation that, when executed, initializes all the global variables in the TensorFlow graph

# Launch the graph
with tf.Session() as sess: # manager that creates a TensorFlow session and automatically manages its
```

```
execution.
# Load initialized variables in current session
 sess.run(init)

 # Fit all training data
 for epoch in range(training_epochs):
# perform gradient descent step
    sess.run(optimizer, feed_dict={X: train_X, y: train_y})

  # Display logs per epoch step
 if (epoch+1) % display_step == 0:
  c = sess.run(cost, feed_dict={X: train_X, y: train_y})

 print("Epoch:{0:6} \t Cost:{1:10.4} \t W:{2:6.4} \t b:{3:6.4}".format(epoch+1, c, sess.run(W), sess.run(b)))
 # Print final parameter values
 print("Optimization Finished!")

 training_cost = sess.run(cost, feed_dict={X: train_X, y: train_y})
 print("Final training cost:", training_cost, "W:", sess.run(W), "b:", sess.run(b), '\n')

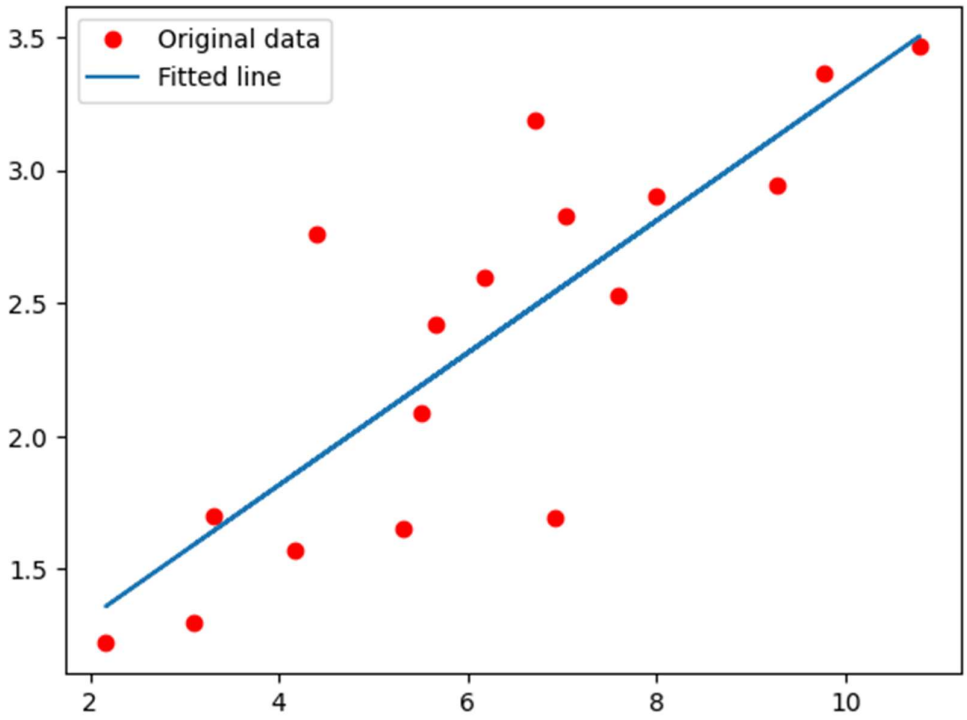
 # Graphic display
 plt.plot(train_X, train_y, 'ro', label='Original data')
 plt.plot(train_X, sess.run(W) * train_X + sess.run(b), label='Fitted line')
 plt.legend()
 plt.show()

 # Testing the model
 testing_cost = sess.run(tf.reduce_sum(tf.square(linear_model - y)) / (2 * test_X.shape[0]), feed_dict={X:
test_X, y: test_y})
 print("Final testing cost:", testing_cost)
 print("Absolute mean square loss difference:", abs(training_cost - testing_cost))

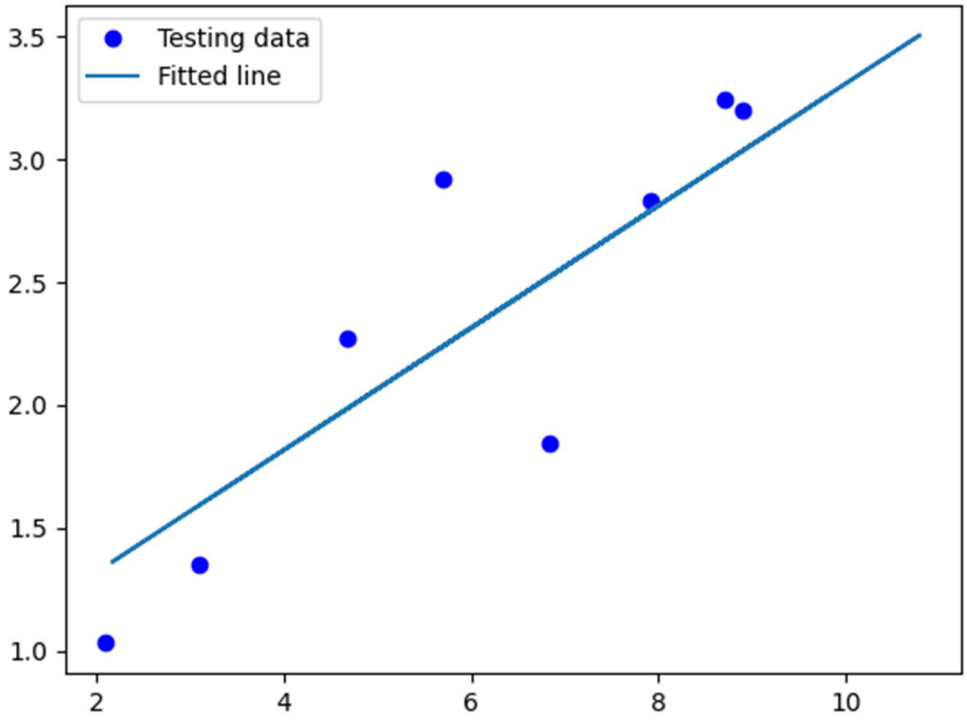
# Display fitted line on test data
 plt.plot(test_X, test_y, 'bo', label='Testing data')
 plt.plot(train_X, sess.run(W) * train_X + sess.run(b), label='Fitted line')
 plt.legend()
 plt.show()
```

OUTPUT:

```
Instructions for updating:
non-resource variables are not supported in the long term
Epoch:  2000    Cost:   0.07696           W:0.2485        b:0.8209
Optimization Finished!
Final training cost: 0.076958984 W: 0.24852164 b: 0.820873
```



Final testing cost: 0.07895341
Absolute mean square loss difference: 0.0019944236



PyTorch:

PyTorch
import torch # developing and training neural network based deep learning models.
import math # extends the list of mathematical functions.
dtype = torch.float

```
device = torch.device("cpu")
# device = torch.device("cuda:0") # Uncomment this to run on GPU

# Create random input and output data
x = torch.linspace(-math.pi, math.pi, 2000, device=device, dtype=dtype)
y = torch.sin(x)

# Randomly initialize weights
a = torch.randn((), device=device, dtype=dtype)
b = torch.randn((), device=device, dtype=dtype)
c = torch.randn((), device=device, dtype=dtype)
d = torch.randn((), device=device, dtype=dtype)
learning_rate = 1e-6

for t in range(2000):
# Forward pass: compute predicted y
  y_pred = a + b * x + c * x ** 2 + d * x ** 3
# Compute and print loss
loss = (y_pred - y).pow(2).sum().item()

if t % 100 == 99:
 print(t, loss)

# Backprop to compute gradients of a, b, c, d with respect to loss
grad_y_pred = 2.0 * (y_pred - y)
grad_a = grad_y_pred.sum()
grad_b = (grad_y_pred * x).sum()
grad_c = (grad_y_pred * x ** 2).sum() # calculates the gradient of some scalar variable c
with respect to the tensor x in PyTorch.
grad_d = (grad_y_pred * x ** 3).sum()

# Update weights using gradient descent
a -= learning_rate * grad_a # updates the value of some scalar variable a using gradient
descent in PyTorch.
b -= learning_rate * grad_b
c -= learning_rate * grad_c
d -= learning_rate * grad_d
print(f'Result: y = {a.item()} + {b.item()} x + {c.item()} x^2 + {d.item()} x^3')
```

OUTPUT:

```
1999 6153.05859375
Result: y = -0.8028205037117004 + 1.1849738359451294 x + 0.47501543164253235 x^2 +
-0.15010419487953186 x^3
```

Conclusion:

Exploring Python libraries like TensorFlow and Keras enables efficient development and deployment of deep learning models with powerful, user-friendly tools. These libraries simplify complex neural network design, making advanced AI accessible for a wide range of applications.

CO's Covered:

CO1. Implement basic neural network models.

Expt. No. 2

Title To implement McCulloch Pitts model for binary logic functions

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 07/08/2025

***Sign:*_____**

Experiment No. 2

Deep learning

Aim: Implement McCulloch Pitts model for binary logic function

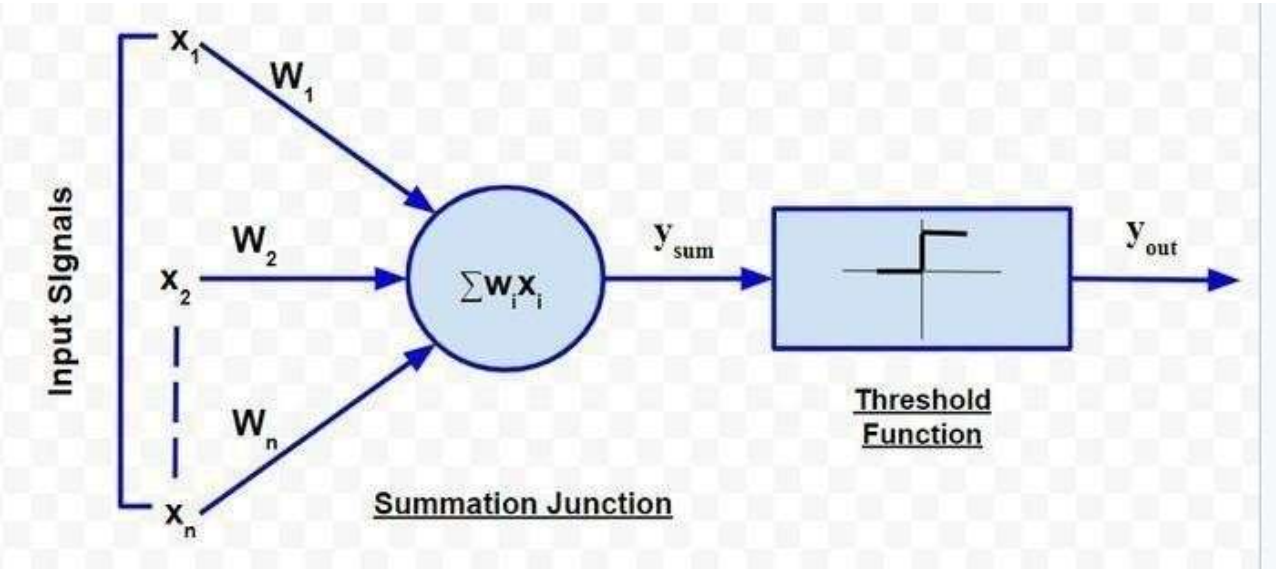
Software: Google collab

Pre-Lab Questions: Explain McCulloch and Pitts model?
The McCulloch-Pitts model is a simple artificial neuron that computes a weighted sum of inputs and produces a binary output using a threshold function.

Post-Lab Questions: Explain simulation of And gate using Mc Culloch -Pitts model
An AND gate in the McCulloch-Pitts model is simulated by assigning weights of 1 to each input and setting the threshold to 2 so output is 1 only if both inputs are 1.

Theory: The McCulloch-Pitts neural model, which was the earliest ANN model, has only two types of inputs — **Excitatory and Inhibitory.** The excitatory inputs have weights of positive magnitude, and the inhibitory weights have weights of negative magnitude. The inputs of the McCulloch-Pitts neuron could be either 0 or 1. It has a threshold function as an activation function. So, the output signal y_{out} is 1 if the input y_{sum} is greater than or equal to a given threshold value, else 0. The diagrammatic representation of the model is as follows

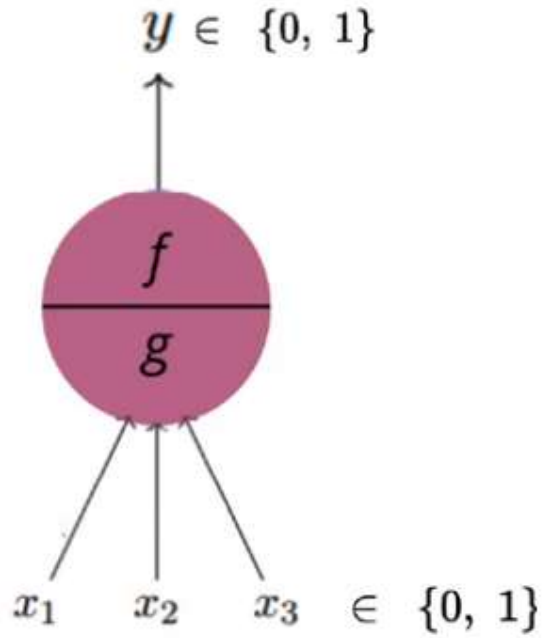
Simple McCulloch-Pitts neurons can be used to design logical operations. For that purpose, the connection weights need to be correctly decided along with the threshold function (rather than the threshold value of the activation function).



Simulation of And Gate using McCulloch Pitts Network

McCulloch-Pitts Neuron Model:

MP Neuron Model introduced by Warren McCulloch and Walter Pitts in 1943. MP neuron model is also known as linear threshold gate model.
Mathematical Model



Fig—2 Simple representation of MP Neuron Model

The function (soma) is actually split into two parts: **g**—The aggregates the inputs to a single numeric value and the function **f** produces the output of this neuron by taking the output of the **g** as the input i,e.. a single value as its argument. The function **f** will output the value 1 if the aggregation performed by the function **g** is greater than some threshold else it will return 0.
The inputs x1, x2, ….. xn for the MP Neuron can only take Boolean values and the inputs can be inhibitory or excitatory. Inhibitory inputs can have maximum effect on the decision-making process of the model. In some cases, inhibitory inputs can influence the final outcome of the model.

$$y = 0 \text{ if any } x_i \text{ is inhibitory, else}$$

$$g(x_1, x_2, ...x_n) = g(x) = \sum_{i=1}^{n} x_i$$

$$y = f(g(x)) = 1 \text{ if } g(x) \geq b$$

$$= 0 \text{ if } g(x) < b$$

Fig—3 Mathematical representation

For example, I can predict my own decision of whether I would like to watch a movie in a nearby IMAX theater tomorrow or not using an MP Neuron Model. All the inputs to the model are boolean i.e., [0,1] and

from the Fig—2 we can see that output from the model will also be boolean. (0—Not going to movie, 1—going to the movie)

Inputs for the above problem could be

- x1—IsRainingTomorrow (Whether it's going to rain tomorrow or not)
- x2—IsScifiMovie (I like science fiction movies)
- x3—IsSickTomorrow (Whether I am going to be sick tomorrow or not depends on any symptoms, eg: fever)
- x4—IsDirectedByNolan (Movie directed by Christopher Nolan or not.) etc….

In this scenario, if x3—IsSickTomorrow is equal to 1, then the output will always be 0. If I am not feeling well on the day of the movie then no matter whoever is the actor or director of the movie, I wouldn't be going for a movie.

Loss Function

Let's take an example of buying a phone based on some features of the features in the binary format. { y—0: Not buying a phone and y—1: buying a phone}



Launch (within 6 months)	0	1	1	0	0	1	0	1	1
Weight (<160g)	1	0	1	0	0	0	1	0	0
Screen size (<5.9 in)	1	0	1	0	1	0	1	0	1
dual sim	1	1	0	0	0	1	0	1	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1
NFC	0	1	1	0	1	0	1	1	1
Radio	1	0	0	1	1	1	0	0	0
Battery(>3500mAh)	0	0	0	1	0	1	0	1	0
Price > 20k	0	1	1	0	0	0	1	1	1
Like (y)	1	0	1	0	1	1	0	1	0

Fig—4: Buying a phone

For each particular phone (observation) with a certain threshold value **b**, using the MP Neuron Model, we can predict the outcome using a condition that the summation of the inputs is greater than b then the predicted value will be 1 or else it will be 0. The loss for the particular observation will be squared difference between the Yactual and Ypredicted.

$$\hat{y} = \sum_{i=1}^{n} x_i \geq b$$

Fig—5: MP Neuron Model for Buying a phone

Similarly, for all the observations, calculate the summation of the squared difference between the Yactual and Ypredicted to get the total loss of the model for a particular threshold value **b**.

$$loss = \sum_i (y_i - \hat{y}_i)^2$$

Fig—6: Loss of the Model

Algorithm:

- The purpose of the learning algorithm is to find out the best value for the parameter **b** so that the loss of the model will be minimum. In the ideal scenario, the loss of the model for the best value of **b** would be zero.
- For n features in the data, the summation we are computing in Fig—5 can take only values between 0 and n because all of our inputs are binary (0 or 1). 0—indicates all the features described in the Fig—4 are off and 1—indicates all the features are on. Therefore the different values the threshold b can take will also vary from 0 to n. As we have only one parameter with a range of values 0 to n, we can use the brute force approach to find the best value of b.
- Initialize the b with a random integer [0,n] For each observation Find the predicted outcome, by using the formula in Fig—5 Calculate the summation of inputs and check whether its greater than or equal to b. If its greater than or equal to b, then the predicted outcome will be 1 or else it will be 0.
- After finding the predicting outcome compute the loss for each observation. Finally, compute the total loss of the model by summing up all the individual losses. Similarly, we can iterate over all the possible values of b and find the total loss of the model. Then we can choose the value of b, such that the loss is minimum.
- Model Evaluation
- After finding the best threshold value b from the learning algorithm, we can evaluate the model on the test data by comparing the predicted outcome and the actual outcome.

Training data

Launch (within 6 months)	0	1	1	0	0	1	0	1	1	0
Weight (<160g)	1	0	1	0	0	0	1	0	0	1
Screen size (<5.9 in)	1	0	1	0	1	0	1	0	1	0
dual sim	1	1	0	0	0	1	0	1	0	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1	0
NFC	0	1	1	0	1	0	1	1	1	0
Radio	1	0	0	1	1	1	0	0	0	0
Battery(>3500mAh)	0	0	0	1	0	1	0	1	0	0
Price > 20k	0	1	1	0	0	0	1	1	1	0
Like? (y)	1	1	1	0	0	1	1	1	0	0
predicted	1	1	0	1	1	1	1	0	0	0

Test data

1	0	0	1
0	1	1	1
0	1	1	1
0	1	0	0
1	0	0	0
0	0	1	0
1	1	1	0
1	1	1	0
0	0	1	0
0	1	0	0
0	1	1	0

Fig—7: Predictions on the test data for b = 5

For evaluation, we will calculate the accuracy score of the model.

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Fig—8: Accuracy metric

For the above-shown test data, the accuracy of the MP neuron model = 75%.

Program & Output

```
import numpy as np

# McCulloch-Pitts Model
class McCullochPittsNeuron:
    def __init__(self, weights, threshold):
        # Initialize with weights and threshold
        self.weights = weights
        self.threshold = threshold

    def activate(self, inputs):
        """
        Activates the neuron using the binary step function.
        inputs: list of inputs
        Returns: 1 if weighted sum is greater than threshold, otherwise 0
        """
        # Weighted sum of inputs
        weighted_sum = np.dot(inputs, self.weights)

        # threshold
        return 1 if weighted_sum >= self.threshold else 0

def AND(inputs):
    weights = [1, 1]  # AND function needs positive weights
    threshold = 2  # Threshold set to 2 for AND
    neuron = McCullochPittsNeuron(weights, threshold)
    return neuron.activate(inputs)

def OR(inputs):
    weights = [1, 1]  # OR function needs positive weights
    threshold = 1  # Threshold set to 1 for OR
    neuron = McCullochPittsNeuron(weights, threshold)
    return neuron.activate(inputs)

def NOT(inputs):
    weights = [-1]  # NOT function uses negative weights
    threshold = -0.5  # Threshold set to -0.5 for NOT
    neuron = McCullochPittsNeuron(weights, threshold)
    return neuron.activate(inputs)
```

```
# AND Gate
print("AND Gate")
print("0 AND 0:", AND([0, 0]))
print("0 AND 1:", AND([0, 1]))
print("1 AND 0:", AND([1, 0]))
print("1 AND 1:", AND([1, 1]))

# OR Gate
print("\nOR Gate")
print("0 OR 0:", OR([0, 0]))
print("0 OR 1:", OR([0, 1]))
print("1 OR 0:", OR([1, 0]))
print("1 OR 1:", OR([1, 1]))

# NOT Gate
print("\nNOT Gate")
print("NOT 0:", NOT([0]))
print("NOT 1:", NOT([1]))
```

OUTPUT:

```
AND Gate
0 AND 0: 0
0 AND 1: 0
1 AND 0: 0
1 AND 1: 1

OR Gate
0 OR 0: 0
0 OR 1: 1
1 OR 0: 1
1 OR 1: 1

NOT Gate
NOT 0: 1
NOT 1: 0
```

Conclusion:

The McCulloch-Pitts model provides a simple framework for implementing binary logic functions by using weighted inputs and a threshold for activation. It effectively models basic gates like AND, OR, and NOT, demonstrating the foundational concept behind neural network computation.

CO's Covered:

CO1. Implement basic neural network models.

Expt. No. 3

Title: Implement Perceptron algorithm to simulate any logic gate.

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 25/08/2025

***Sign:*_____**

Experiment 3

Aim: ***Implement Perceptron algorithm to simulate any logic gate.***

Software: Google Collab

Pre Lab-Questions:

What do you understand by a Perceptron Neuron?

A perceptron neuron is a computational model that sums weighted inputs, applies a bias, and passes the result through an activation function to produce an output.

Post lab Questions:

What are the components of Single Layer Perceptron?

The components of a single-layer perceptron are inputs, weights, bias, summation function, activation function, and output.

Theory:

Overview

Perceptron is a linear supervised machine learning algorithm. It is used for binary classification. This article will introduce you to a very important binary classifier, the perceptrons, which forms the basis for the most popular machine learning models nowadays – the neural networks.

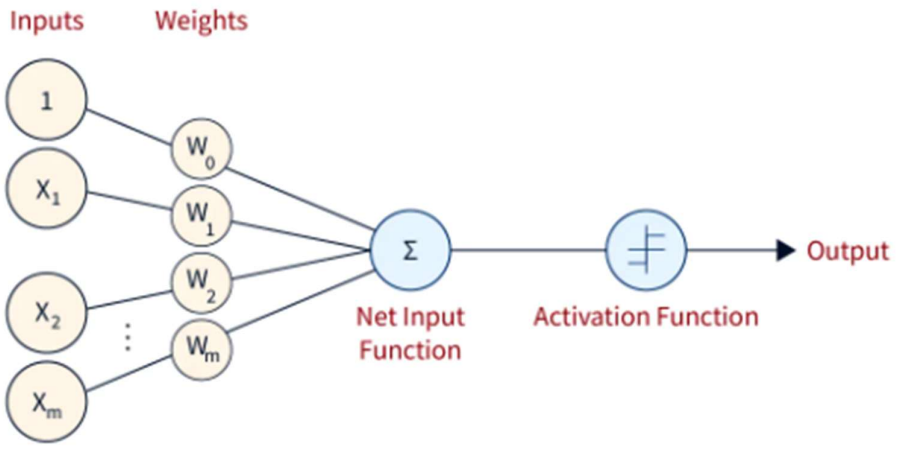
Introduction

Perceptron Learning Algorithm is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence. The perceptron learning algorithm is treated as the most straightforward Artificial Neural network. It is a supervised learning algorithm of binary classifiers. Hence, it is a single-layer neural network with four main parameters, **i.e.**, input values, weights and Bias, net sum, and an activation function.

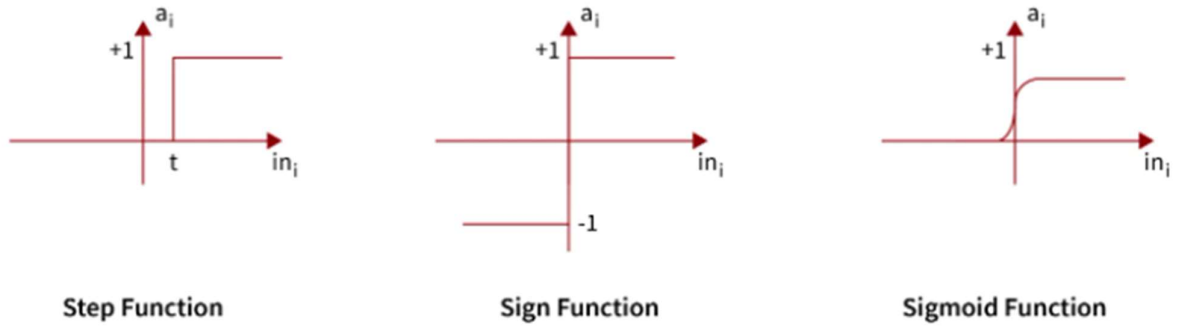
Working:

Basic Components of Perceptron

Frank Rosenblatt invented the perceptron learning algorithm.



It is a binary classifier and consists of three main components. These are:



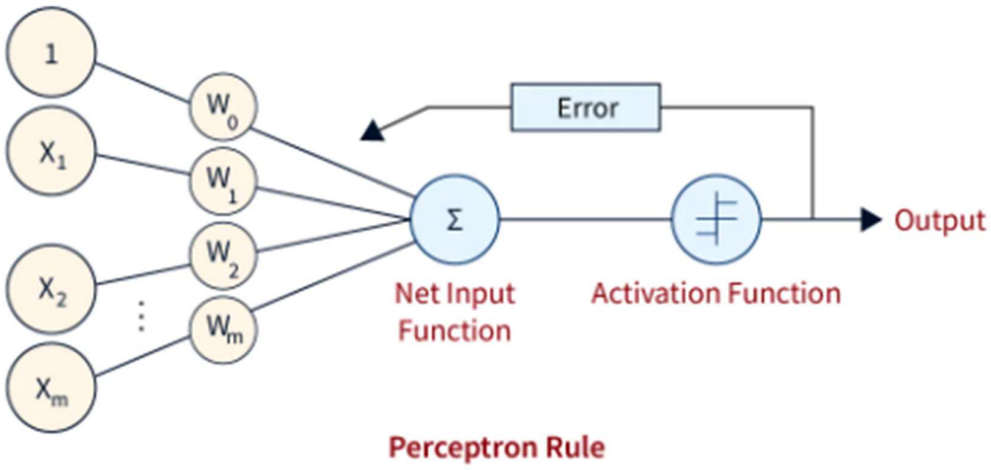
Types of Perceptron Models

Based on the number of layers, perceptron is broadly classified into two major categories:

1. **Single Layer Perceptron Model:**
   It is the simplest Artificial Neural Network (ANN) model. A single-layer perceptron model consists of a feed-forward network and includes a threshold transfer function for thresholding on the Output. The main objective of the single-layer perceptron model is to classify linearly separable data with binary labels.
2. **Multi-Layer Perceptron Model:**
   The multi-layer perceptron learning algorithm has the same structure as a single-layer perceptron but consists of an additional one or more hidden layers, unlike a single-layer perceptron, which consists of a single hidden layer. The distinction between these two types of perceptron models is shown in the Figure below.

Algorithm:

An overview of this algorithm is illustrated in the following Figure:



1. **Input Nodes or Input Layer:** Primary component of Perceptron learning algorithm, which accepts the initial input data into the model. Each input node contains an actual value.
2. **Weight and Bias:** The weight parameter represents the strength of the connection between units. Bias

can be considered as the line of intercept in a linear equation.

3. **Activation Function:** Final and essential components help determine whether the neuron will fire. The activation function can be primarily considered a step function. There are various types of activation functions used in a perceptron learning algorithm. Some of them are the sign function, step function, sigmoid function, etc.

Perceptron Function

Perceptron learning algorithm function f(x)$f(x)$ is represented as the product of the input vector (x) and the learned weight vector (w). In mathematical notion, it can be described as:
f(x)=1,ifw.x+b>0$f(x)=1,ifw.x+b>0$ f(x)=0,otherwise$f(x)=0,otherwise$

Where–

- w represents the weight vector which consists of a set of real-valued weights.
- b represents the bias vector.
- x represents the input vector which consists of the input feature values.

Geometry of the Solution Space

In the previous section, we learned about the weight update rules for the perceptron learning algorithm. We have already established that when x belongs to P, we want w.x > 0. That means that the angle between w and x should be less than 90 because the cosine of the slope is proportional to the dot product.

So, whatever the w vector may be, as long as it makes an angle less than 90 degrees with the positive example data vectors (x ∈∈ P) and an angle more than 90 degrees with the negative example data vectors (x ∈∈ N), we are cool.

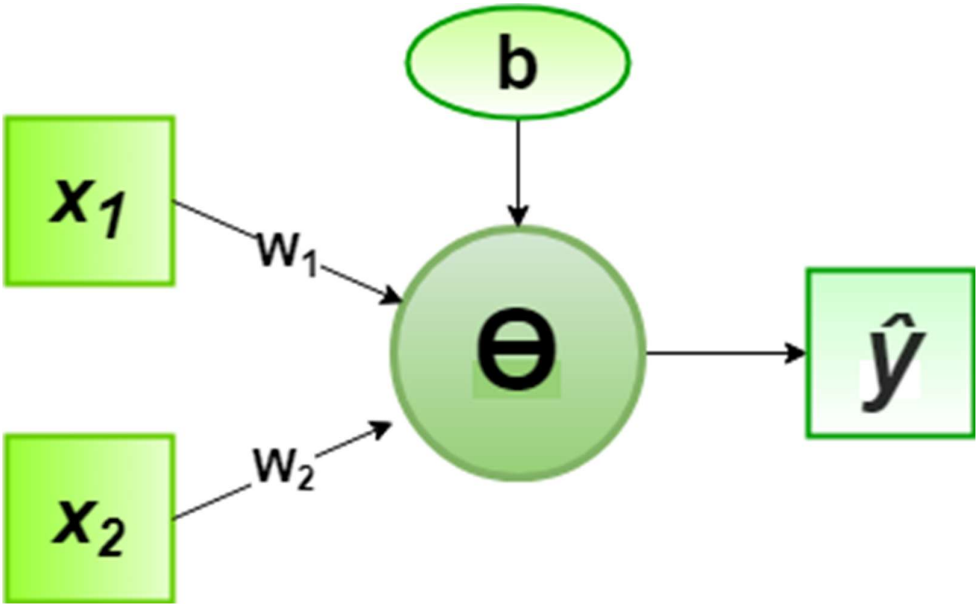
In the field of Machine Learning, the Perceptron is a Supervised Learning Algorithm for binary classifiers. The Perceptron Model implements the following function:

$$\begin{aligned}\hat{y} &= \Theta\left(w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b\right) \\ &= \Theta(\mathbf{w} \cdot \mathbf{x} + b) \\ \text{where } \Theta(v) &= \begin{cases} 1 & \text{if } v \geqslant 0 \\ 0 & \text{otherwise} \end{cases}\end{aligned}$$

X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

Now for the corresponding weight vector
$w : (w_1, w_2)$
of the input vector
$x : (x_1, x_2)$
, the associated Perceptron Function can be defined as:

$$\hat{y} = \Theta\left(w_1 x_1 + w_2 x_2 + b\right)$$



For the implementation, considered weight parameters are
$w_1 = 1, w_2 = 1$
and the bias parameter is
$b = -1.5$

Program & Output:

```
Import numpy as np

def unitStep(v):
  if v>=0:
    return 1
  else:
    return 0

def perceptronModel(x,w,b):
  v = np.dot(w, x) + b
  y = unitStep(v)
  return y

def AND_logic(x):
  w = np.array([1,1])
  b = -1.5
  return perceptronModel(x,w,b)


test1 = np.array([0, 1])
test2 = np.array([1, 1])
test3 = np.array([0, 0])
test4 = np.array([1, 0])

print(f"AND(0, 1) = {AND_logic(test1)}")
print(f"AND(1, 1) = {AND_logic(test2)}")
print(f"AND(0, 0) = {AND_logic(test3)}")
print(f"AND(1, 0) = {AND_logic(test4)}")
```

OUTPUT:

```
AND(0, 1) = 0
AND(1, 1) = 1
AND(0, 0) = 0
AND(1, 0) = 0
```

Conclusion:

The Perceptron algorithm effectively simulates logic gates like AND by learning the appropriate weights and threshold through training. This demonstrates how simple neural networks can model basic binary functions.

CO's Covered:

CO1. Implement basic neural network models.

Expt. No. 4

Title: Implement Multilayer Perceptron algorithm to simulate XOR gate.

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 25/08/2025

Sign: _____

Experiment 4

Aim: ***Implement Multilayer Perceptron algorithm to simulate XOR gate.***

Software: Google Collab

Pre Lab-Questions:

What is Multilayer Perceptron?

A multilayer perceptron is a feedforward neural network with input, hidden, and output layers that use nonlinear activation functions to learn complex patterns.

Post lab Questions:

What is the working of simulating XOR gate using Multilayer Perceptron?

A multilayer perceptron simulates an XOR gate by using hidden neurons to combine inputs nonlinearly so the network outputs 1 only for differing inputs.

Theory:

Multilayer Perceptrons

A multilayer perceptron is a type of feedforward neural network consisting of fully connected neurons with a nonlinear kind of activation function. It is widely used to distinguish data that is not linearly separable.

MLPs have been widely used in various fields, including image recognition, natural language processing, and speech recognition, among others. Their flexibility in architecture and ability to approximate any function under certain conditions make them a fundamental building block in deep learning and neural network research. Let's take a deeper dive into some of its key concepts.

Input layer

The input layer consists of nodes or neurons that receive the initial input data. Each neuron represents a feature or dimension of the input data. The number of neurons in the input layer is determined by the dimensionality of the input data.

Hidden layer

Between the input and output layers, there can be one or more layers of neurons. Each neuron in a hidden layer receives inputs from all neurons in the previous layer (either the input layer or another hidden layer) and produces an output that is passed to the next layer. The number of hidden layers and the number of neurons in each hidden layer are hyperparameters that need to be determined during the model design phase.

Output layer

This layer consists of neurons that produce the final output of the network. The number of neurons in the output layer depends on the nature of the task. In binary classification, there may be either one or two neurons depending on the activation function and representing the probability of belonging to one class; while in multi-class classification tasks, there can be multiple neurons in the output layer.

Weights

Neurons in adjacent layers are fully connected to each other. Each connection has an associated weight, which determines the strength of the connection. These weights are learned during the training process.

Bias neurons

In addition to the input and hidden neurons, each layer (except the input layer) usually includes a bias neuron that provides a constant input to the neurons in the next layer. Bias neurons have their

own weight associated with each connection, which is also learned during training.
The bias neuron effectively shifts the activation function of the neurons in the subsequent layer, allowing the network to learn an offset or bias in the decision boundary. By adjusting the weights connected to the bias neuron, the MLP can learn to control the threshold for activation and better fit the training data.
Note: It is important to note that in the context of MLPs, bias can refer to two related but distinct concepts: bias as a general term in machine learning and the bias neuron (defined above). In general machine learning, bias refers to the error introduced by approximating a real-world problem with a simplified model. Bias measures how well the model can capture the underlying patterns in the data. A high bias indicates that the model is too simplistic and may underfit the data, while a low bias suggests that the model is capturing the underlying patterns well.

Activation function

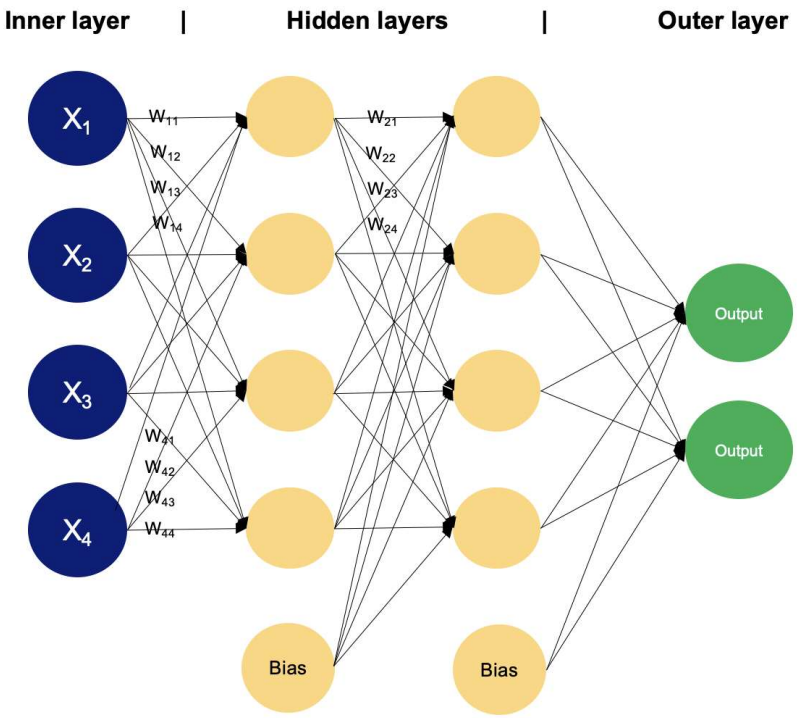
Typically, each neuron in the hidden layers and the output layer applies an activation function to its weighted sum of inputs. Common activation functions include sigmoid, tanh, ReLU (Rectified Linear Unit), and softmax. These functions introduce nonlinearity into the network, allowing it to learn complex patterns in the data.

Working:

Training with backpropagation

MLPs are trained using the backpropagation algorithm, which computes gradients of a loss function with respect to the model's parameters and updates the parameters iteratively to minimize the loss.

Workings of a Multilayer Perceptron: Layer by Layer



In a multilayer perceptron, neurons process information in a step-by-step manner, performing computations that involve weighted sums and nonlinear transformations. Let's walk layer by layer to see the magic that goes within.

Input layer

- The input layer of an MLP receives input data, which could be features extracted from the input samples in a dataset. Each neuron in the input layer represents one feature.
- Neurons in the input layer do not perform any computations; they simply pass the input values to the neurons in the first hidden layer.

Hidden layers

- The hidden layers of an MLP consist of interconnected neurons that perform computations on the input data.
- Each neuron in a hidden layer receives input from all neurons in the previous layer. The inputs are multiplied by corresponding weights, denoted as w. The weights determine how much influence the input from one neuron has on the output of another.
- In addition to weights, each neuron in the hidden layer has an associated bias, denoted as b. The bias provides an additional input to the neuron, allowing it to adjust its output threshold. Like weights, biases are learned during training.
- For each neuron in a hidden layer or the output layer, the weighted sum of its inputs is computed. This involves multiplying each input by its corresponding weight, summing up these products, and adding the bias:

$$\textbf{Weighted Sum} = \sum_{i=1}^{n} (w_i * x_i) + b$$

Where n is the total number of input connections, w_i is the weight for the i-th input, and x_i is the i-th input value.

- The weighted sum is then passed through an activation function, denoted as f. The activation function introduces nonlinearity into the network, allowing it to learn and represent complex relationships in the data. The activation function determines the output range of the neuron and its behavior in response to different input values. The choice of activation function depends on the nature of the task and the desired properties of the network.

Output layer

- The output layer of an MLP produces the final predictions or outputs of the network. The number of neurons in the output layer depends on the task being performed (e.g., binary classification, multi-class classification, regression).
- Each neuron in the output layer receives input from the neurons in the last hidden layer and applies an activation function. This activation function is usually different from those used in the hidden layers and produces the final output value or prediction.
- During the training process, the network learns to adjust the weights associated with each neuron's inputs to minimize the discrepancy between the predicted outputs and the true target values in the training data. By adjusting the weights and learning the appropriate activation functions, the network learns to approximate complex patterns and relationships in the data, enabling it to make accurate predictions on new, unseen samples.
- This adjustment is guided by an optimization algorithm, such as stochastic gradient descent (SGD), which computes the gradients of a loss function with respect to the weights and updates the weights iteratively.

Algorithm :

In the field of Machine Learning, the Perceptron is a Supervised Learning Algorithm for binary classifiers. The Perceptron Model implements the following function:

$$\hat{y} = \Theta\left(w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b\right)$$
$$= \Theta(\mathbf{w} \cdot \mathbf{x} + b)$$
$$\text{where } \Theta(v) = \begin{cases} 1 & \text{if } v \geqslant 0 \\ 0 & \text{otherwise} \end{cases}$$

For a particular choice of the weight vector $\boldsymbol{w}$ and bias parameter $\boldsymbol{b}$, the model predicts output $\hat{\boldsymbol{y}}$ for the corresponding input vector $\boldsymbol{x}$. XOR logical function truth table for ***2-bit binary variables***, i.e, the input vector $\boldsymbol{x} : (\boldsymbol{x_1}, \boldsymbol{x_2})$ and the corresponding output $\boldsymbol{y}$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

We can observe that, $XOR(x_1, x_2) = AND(NOT(AND(x_1, x_2)), OR(x_1, x_2))$ **Designing the Perceptron Network:**

1. **Step1:** Now for the corresponding weight vector
   $w : (w_1, w_2)$
   of the input vector
   $x : (x_1, x_2)$
   to the AND and OR node, the associated Perceptron Function can be defined as:

$$\hat{y}_1 = \Theta\left(w_1 x_1 + w_2 x_2 + b_{AND}\right)$$

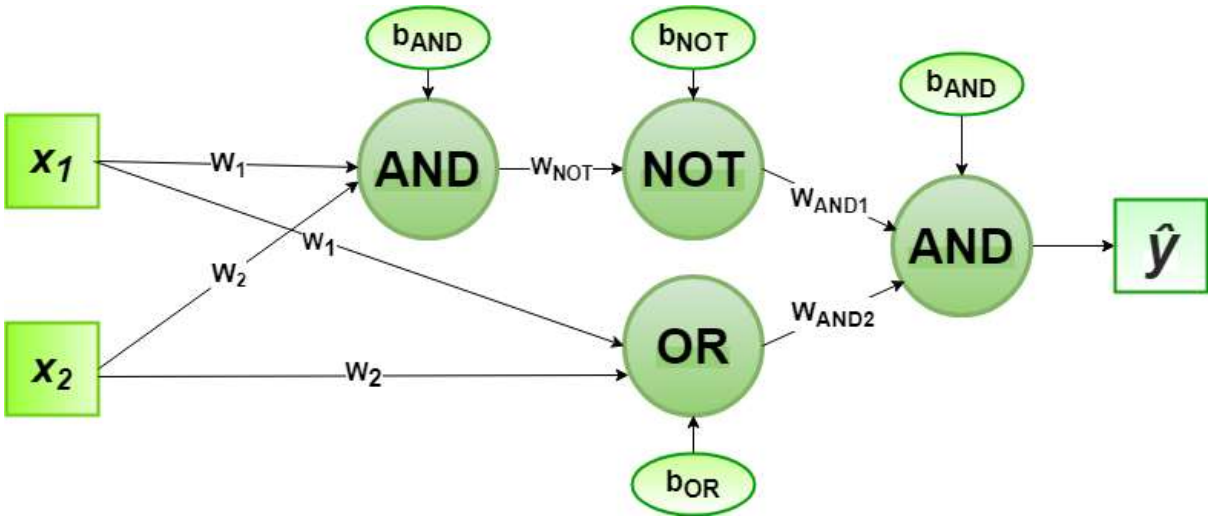
2. **Step2:** The output
   $(\hat{y}_1)$
   from the AND node will be inputted to the NOT node with weight
   w_{NOT}
   and the associated Perceptron Function can be defined as:

$$\hat{y}_3 = \Theta\left(w_{NOT}\hat{y}_1 + b_{NOT}\right)$$

3. **Step3:** The output
   $(\hat{y}_2)$
   from the OR node and the output
   $(\hat{y}_3)$

from NOT node as mentioned in Step2 will be inputted to the AND node with weight (w_{AND1}, w_{AND2}). Then the corresponding output $\hat{y}$ is the final output of the XOR logic function. The associated Perceptron Function can be defined as:

$$\hat{y} = \Theta\left(w_{AND1}\hat{y}_3 + w_{AND2}\hat{y}_2 + b_{AND}\right)$$



For the implementation, the weight parameters are considered to be and the bias parameters are $b_{AND} = -1.5, b_{OR} = -0.5, b_{NOT} = 0.5$. Python

Program & Output

```
Import numpy as np

def unitStep(v):
  if v>=0:
    return 1
  else:
    return 0

def perceptronModel(x,w,b):
  v = np.dot(w, x) + b
  y = unitStep(v)
  return y

def NOT_logicFunction(x):
  wNOT = -1
  bNOT = 0.5
  return perceptronModel(x, wNOT, bNOT)

def AND_logicFunction(x):
  w = np.array([1, 1])
  bAND = -1.5
  return perceptronModel(x, w, bAND)

def OR_logicFunction(x):
  w = np.array([1, 1])
  bOR = -0.5
  return perceptronModel(x, w, bOR)

def XOR_logic(x):
  y1 = AND_logicFunction(x)
  y2 = OR_logicFunction(x)
  y3 = NOT_logicFunction(y1)
  final_x = np.array([y2, y3])
  finalOutput = AND_logicFunction(final_x)
  return finalOutput

test1 = np.array([0, 1])
test2 = np.array([1, 1])
test3 = np.array([0, 0])
test4 = np.array([1, 0])
```

```
print(f"XOR(0, 1) = {XOR_logic(test1)}")
print(f"XOR(1, 1) = {XOR_logic(test2)}")
print(f"XOR(0, 0) = {XOR_logic(test3)}")
print(f"XOR(1, 0) = {XOR_logic(test4)}")
```

OUTPUT:

```
XOR(0, 1) = 1
XOR(1, 1) = 0
XOR(0, 0) = 0
XOR(1, 0) = 1
```

Conclusion:
The Multilayer Perceptron successfully simulates the XOR gate by using hidden layers to capture nonlinear relationships that a single perceptron cannot. This highlights the power of deep networks in solving complex, non-linearly separable problems.

CO's Covered:
CO1. Implement basic neural network model

Expt. No. 5

Title: *Apply any one of stochastic gradient descent learning algorithms to learn the parameters of supervised single layer feed forward neural network*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 01/09/2025

Sign: _____

Experiment 5

Aim: Apply any one of the stochastic gradient descent algorithms to learn the parameters of a supervised single layer feed forward network.

Software: Google Collab

Prelab Question:

Explain the use of gradient descent algorithm in learning the parameters?

Gradient descent updates model parameters iteratively by moving them in the direction that minimizes the loss function.

Post Lab Question:

Explain various types of gradient descent algorithms?

Types of gradient descent include batch, stochastic, and mini-batch, differing in how many training samples are used per update.

Theory:

Gradient Descent is an iterative optimization process that searches for an objective function's optimum value (Minimum/Maximum). It is one of the most used methods for changing a model's parameters in order to reduce a cost function in machine learning projects.
The primary goal of gradient descent is to identify the model parameters that provide the maximum accuracy on both training and test datasets. In gradient descent, the gradient is a vector pointing in the general direction of the function's steepest rise at a particular point. The algorithm might gradually drop towards lower values of the function by moving in the opposite direction of the gradient, until reaching the minimum of the function.

Types of Gradient Descent:
Typically, there are three types of Gradient Descent:

1. Batch Gradient Descent
2. Stochastic Gradient Descent
3. Mini-batch Gradient Descent

Algorithm:

Stochastic Gradient Descent Algorithm

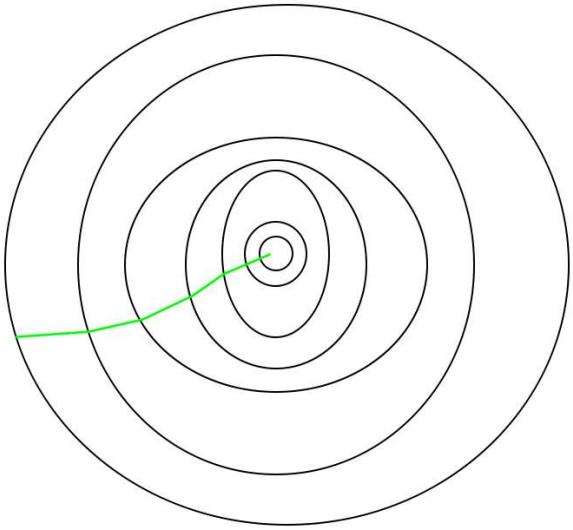
- Initialization: Randomly initialize the parameters of the model.
- Set Parameters: Determine the number of iterations and the learning rate (alpha) for updating the parameters.
- Stochastic Gradient Descent Loop: Repeat the following steps until the model converges or reaches the maximum number of iterations:
    - Shuffle the training dataset to introduce randomness.
    - Iterate over each training example (or a small batch) in the shuffled order.
    - Compute the gradient of the cost function with respect to the model parameters using the current training
      example (or batch).
    - Update the model parameters by taking a step in the direction of the negative gradient, scaled by the learning rate.

- Evaluate the convergence criteria, such as the difference in the cost function between iterations of the gradient.

- Return Optimized Parameters: Once the convergence criteria are met or the maximum number of iterations is reached, return the optimized model parameters.

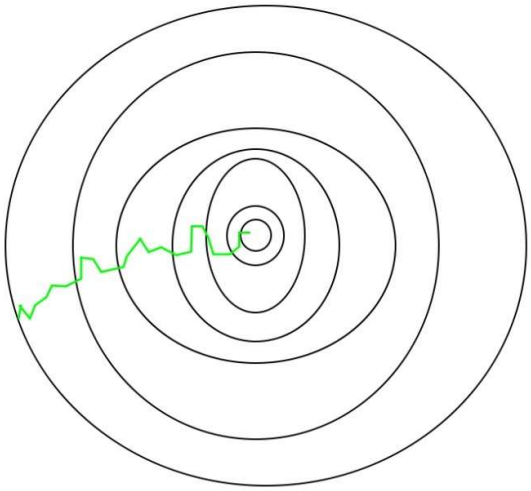
In SGD, since only one sample from the dataset is chosen at random for each iteration, the path taken by the algorithm to reach the minima is usually noisier than your typical Gradient Descent algorithm. But that doesn't matter all that much because the path taken by the algorithm does not matter, as long as we reach the minimum and with a significantly shorter training time.

The path taken by Batch Gradient Descent is shown below:



Batch gradient optimization path

A path taken by Stochastic Gradient Descent looks as follows –



stochastic gradient optimization path

One thing to be noted is that, as SGD is generally noisier than typical Gradient Descent, it usually took a higher number of iterations to reach the minima, because of the randomness in its descent. Even though it requires a higher number of iterations to reach the minima than typical Gradient Descent, it is still computationally much less expensive than typical Gradient Descent. Hence, in most scenarios, SGD is preferred over Batch Gradient Descent for optimizing a learning algorithm.

Program & Output

```
import numpy as np

class SGD:
    def __init__(self, lr=0.01, epochs=50, batch_size=32, tol=1e-3):
        self.learning_rate = lr
        self.epochs = epochs
        self.batch_size = batch_size
        self.tolerance = tol
        self.weights = None
        self.bias = None

    def predict(self, X):
        return np.dot(X, self.weights) + self.bias

    def mean_squared_error(self, y_true, y_pred):
        return np.mean((y_true - y_pred) ** 2)

    def gradient(self, X_batch, y_batch):
        y_pred = self.predict(X_batch)
        error = y_pred - y_batch
        gradient_weights = np.dot(X_batch.T, error) / X_batch.shape[0]
        gradient_bias = np.mean(error)
        return gradient_weights, gradient_bias

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.random.randn(n_features)
        self.bias = np.random.randn()

        for epoch in range(self.epochs):
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            for i in range(0, n_samples, self.batch_size):
                X_batch = X_shuffled[i:i+self.batch_size]
                y_batch = y_shuffled[i:i+self.batch_size]

                gradient_weights, gradient_bias = self.gradient(X_batch, y_batch)
                self.weights -= self.learning_rate * gradient_weights
                self.bias -= self.learning_rate * gradient_bias

            if epoch % 5 == 0:
                y_pred = self.predict(X)
```

```
            loss = self.mean_squared_error(y, y_pred)
            print(f"Epoch {epoch}: Loss {loss}")

        if np.linalg.norm(gradient_weights) < self.tolerance:
            print("Convergence reached.")
            break

    return self.weights, self.bias
```

Implementation

```
# Create random dataset with 100 rows and 5 columns
X = np.random.randn(100, 5)

# create corresponding target value by adding random noise in the dataset
y = np.dot(X, np.array([1, 2, 3, 4, 5])) + np.random.randn(100) * 0.1

# Create an instance of the SGD class
model = SGD(lr=0.01, epochs=300,
            batch_size=32, tol=1e-3)
w,b=model.fit(X,y)
# Predict using predict method from model
y_pred = w*X+b
```

OUTPUT:

```
Epoch 10: Loss 36.83902102412165
Epoch 15: Loss 24.319899960263573
Epoch 20: Loss 16.282765092319295
Epoch 25: Loss 10.321864503922669
Epoch 30: Loss 6.999428232341828
Epoch 35: Loss 4.827150728210429
Epoch 40: Loss 3.232828368260741
Epoch 45: Loss 2.1268789039079037
Epoch 50: Loss 1.4218184556972822
Epoch 55: Loss 0.9412865360555527
Epoch 60: Loss 0.6470548353796197
Epoch 65: Loss 0.449208824630044
Epoch 70: Loss 0.3095440661275809
Epoch 75: Loss 0.21041296761840184
Epoch 80: Loss 0.14413093126438314
Epoch 85: Loss 0.09590229924128563
Epoch 90: Loss 0.06832192284566814
Epoch 95: Loss 0.047815411982855105
Epoch 100: Loss 0.03433093863262353
```

```
Epoch 105: Loss 0.026396803814307 19
Epoch 110: Loss 0.020593096179001038
Epoch 115: Loss 0.016366146753255473
Epoch 120: Loss 0.013777339812725916
Epoch 125: Loss 0.012166408082821854
Epoch 130: Loss 0.011013998433223817
Epoch 135: Loss 0.010368027279322917
Epoch 140: Loss 0.009797246165275659
Epoch 145: Loss 0.009571011820295308
Epoch 150: Loss 0.009319837954281398
Epoch 155: Loss 0.009153042360763553
Epoch 160: Loss 0.009119868400032233
Epoch 165: Loss 0.009054382919628255
Epoch 170: Loss 0.009003554755430182
Epoch 175: Loss 0.008977847182312888
Epoch 180: Loss 0.008962163645710704
Epoch 185: Loss 0.008943006211655518
Epoch 190: Loss 0.008937114565367756
Epoch 195: Loss 0.008915529785911494
Epoch 200: Loss 0.008911057690807037
Epoch 205: Loss 0.00891031219151988
Epoch 210: Loss 0.008918415587164916
Epoch 215: Loss 0.008938118546777096
Epoch 220: Loss 0.008916932563959054
Epoch 225: Loss 0.008912583172415715
Epoch 230: Loss 0.008906888366635033
Epoch 235: Loss 0.008907746160737858
Epoch 240: Loss 0.00891786860580833
Epoch 245: Loss 0.008917162678634237
Epoch 250: Loss 0.008916430547949796
Epoch 255: Loss 0.008916129320380436
Epoch 260: Loss 0.008906114575013172
Epoch 265: Loss 0.008934911442453798
Epoch 270: Loss 0.008925626934384253
Epoch 275: Loss 0.008907678694029194
Epoch 280: Loss 0.00890465435804658
Epoch 285: Loss 0.00890956669364532
Epoch 290: Loss 0.008910838782186362
Epoch 295: Loss 0.0089094138027093
```

Conclusion:

Applying Stochastic Gradient Descent to a supervised single-layer feedforward neural network enables efficient optimization of weights by iteratively minimizing the error. This approach improves the model's learning speed and generalization on training data.

CO's Covered:

CO1. Implement basic neural network models.
CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance

Expt. No. 6

Title: Apply Mini Batch GD on a supervised single layer feed forward neural network.

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 01/09/2025

Sign: _____

Experiment 6

Aim: Apply Mini-batch gradient descent algorithms to learn the parameters of a supervisedsingle layer feed forward network.

Software: Google Collab

Prelab Question:

Explain the use of gradient descent algorithm in learning the parameters?

Gradient descent is used to optimize model parameters by minimizing the loss function through iterative updates.

Post Lab Question:

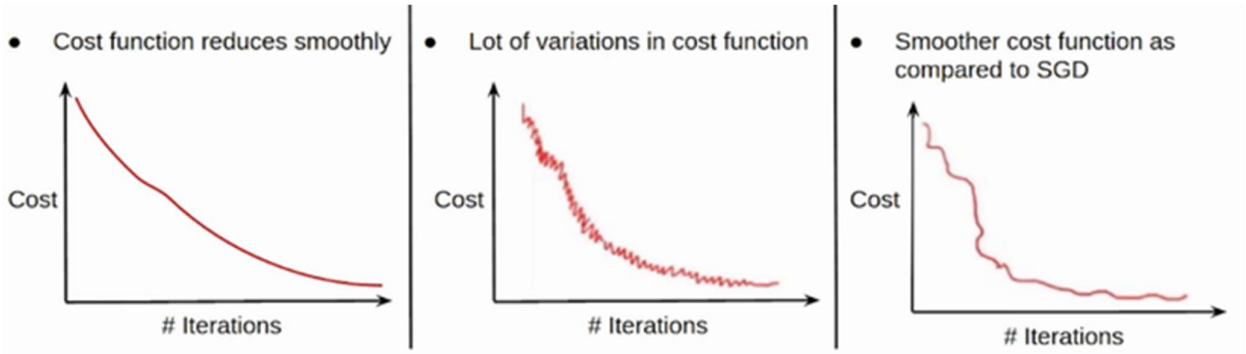
Explain various types of gradient descent algorithms?

Gradient descent types are batch, stochastic, and mini-batch, based on the number of samples used per update.

Theory:

In machine learning, gradient descent is an optimization technique used for computing the model parameters (coefficients and bias) for algorithms like linear regression, logistic regression, neural networks, etc. In this technique, we repeatedly iterate through the training set and update the model parameters in accordance with the gradient of the error with respect to the training set. Depending on the number of training examples considered in updating the model parameters, we have 3-types of gradient descents:

1. **Batch Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to the entire training set
2. **Stochastic Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to a single training example
3. **Mini-Batch Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to a subset of the training set
4. Thus, mini-batch gradient descent makes a compromise between the speedy convergence and the noise associated with gradient update which makes it a more flexible and robust algorithm.



Algorithm:

Let theta = model parameters and max_iters = number of epochs. for itr = 1, 2, 3, …, max_iters: for mini_batch (X_mini, y_mini):

- Forward Pass on the batch X_mini:
  - Make predictions on the mini-batch
  - Compute error in predictions (J(theta)) with the current values of the parameters
- Backward Pass:
  - Compute gradient(theta) = partial derivative of J(theta) w.r.t. theta
- Update parameters:
  - theta = theta – learning_rate*gradient(theta)

Program & Output

```
# importing dependencies
import numpy as np
import matplotlib.pyplot as plt

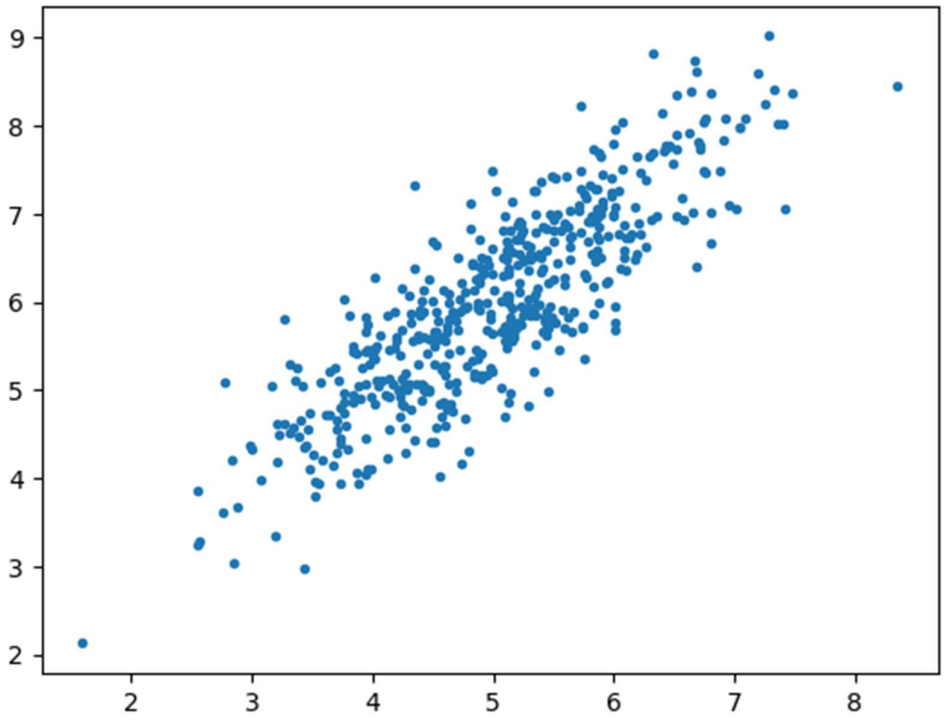
# creating data
mean = np.array([5.0, 6.0])
cov = np.array([[1.0, 0.95], [0.95, 1.2]])
data = np.random.multivariate_normal(mean, cov, 6000)

# visualising data
plt.scatter(data[:500, 0], data[:500, 1], marker='.')
plt.show()

# train-test-split
data = np.hstack((np.ones((data.shape[0], 1)), data))

split_factor = 0.90
split = int(split_factor * data.shape[0])

X_train = data[:split, :-1]
y_train = data[:split, -1].reshape((-1, 1))
X_test = data[split:, :-1]
y_test = data[split:, -1].reshape((-1, 1))
```




```
# linear regression using "mini-batch" gradient descent
# function to compute hypothesis / predictions

def hypothesis(X, theta):
  return np.dot(X, theta)

# function to compute gradient of error function w.r.t. theta

def gradient(X, y, theta):
  h = hypothesis(X, theta)
  grad = np.dot(X.transpose(), (h - y))
  return grad

# function to compute the error for current values of theta

def cost(X, y, theta):
  h = hypothesis(X, theta)
  J = np.dot((h - y).transpose(), (h - y))
  J /= 2
  return J[0]

# function to create a list containing mini-batches

def create_mini_batches(X, y, batch_size):
  mini_batches = []
  data = np.hstack((X, y))
  np.random.shuffle(data)
  n_minibatches = data.shape[0] // batch_size
```

```
  i = 0

  for i in range(n_minibatches + 1):
    mini_batch = data[i * batch_size:(i + 1)*batch_size, :]
    X_mini = mini_batch[:, :-1]
    Y_mini = mini_batch[:, -1].reshape((-1, 1))
    mini_batches.append((X_mini, Y_mini))
  if data.shape[0] % batch_size != 0:
    mini_batch = data[i * batch_size:data.shape[0]]
    X_mini = mini_batch[:, :-1]
    Y_mini = mini_batch[:, -1].reshape((-1, 1))
    mini_batches.append((X_mini, Y_mini))
  return mini_batches

# function to perform mini-batch gradient descent

def gradientDescent(X, y, learning_rate=0.001, batch_size=32):
  theta = np.zeros((X.shape[1], 1))
  error_list = []
  max_iters = 3
  for itr in range(max_iters):
    mini_batches = create_mini_batches(X, y, batch_size)
    for mini_batch in mini_batches:
      X_mini, y_mini = mini_batch
      theta = theta - learning_rate * gradient(X_mini, y_mini, theta)
      error_list.append(cost(X_mini, y_mini, theta))

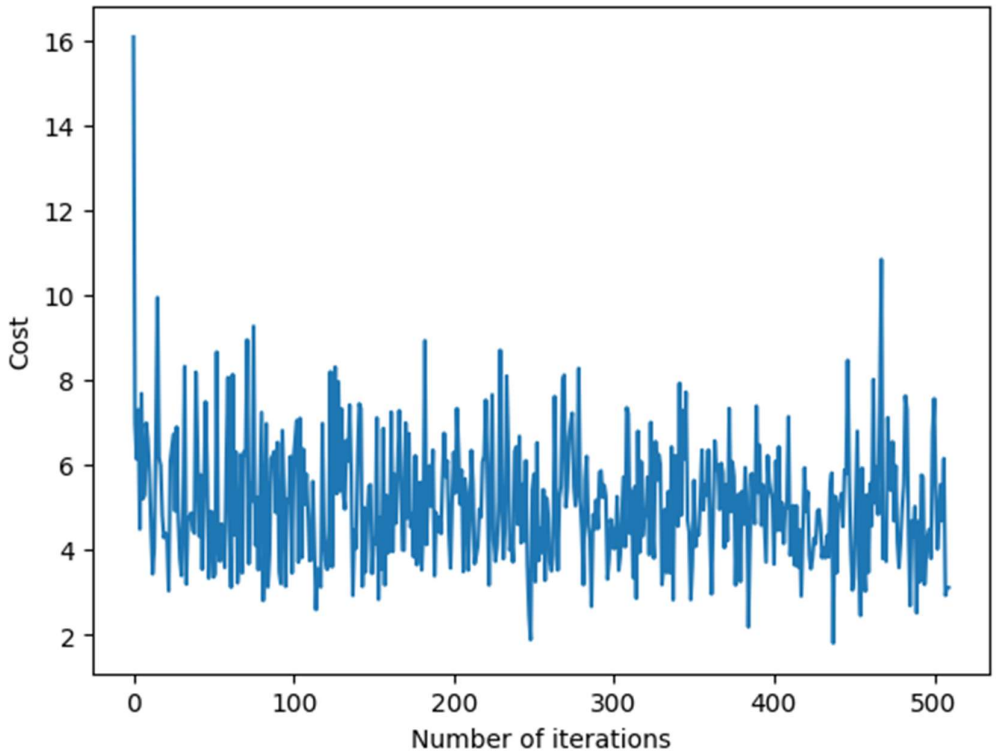
  return theta, error_list


theta, error_list = gradientDescent(X_train, y_train)
print("Bias = ", theta[0])
print("Coefficients = ", theta[1:])

# visualising gradient descent
plt.plot(error_list)
plt.xlabel("Number of iterations")
plt.ylabel("Cost")
plt.show()
```

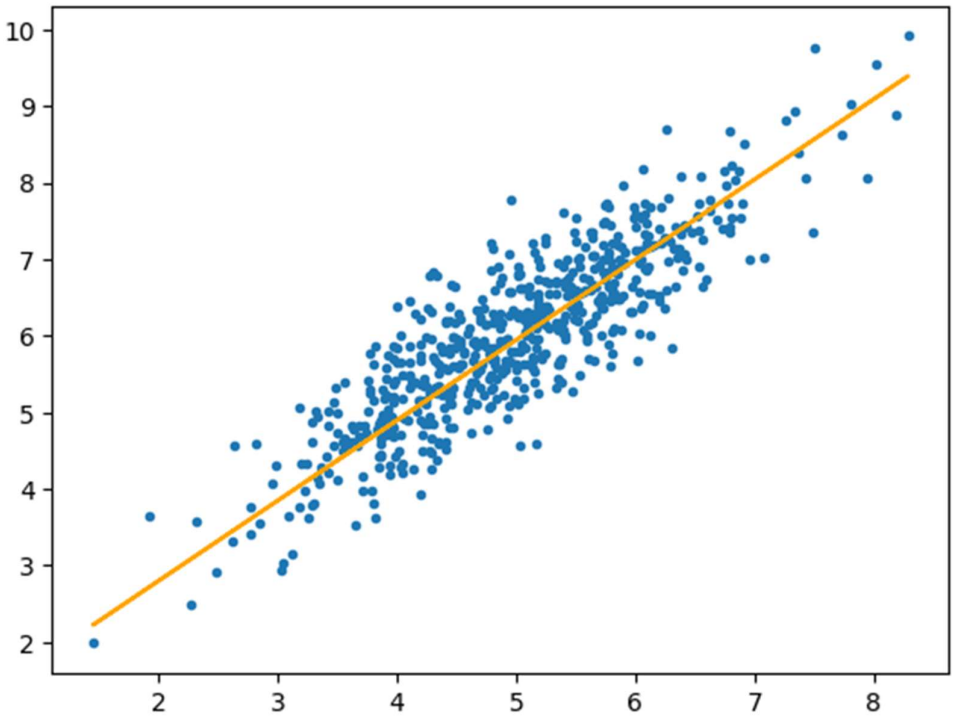
OUTPUT:

```
Bias =  [0.69901842]
Coefficients =  [[1.04894801]]
```




```
# predicting output for X_test
y_pred = hypothesis(X_test, theta)
plt.scatter(X_test[:, 1], y_test[:, ], marker='.')
plt.plot(X_test[:, 1], y_pred, color='orange')
plt.show()

# calculating error in predictions
error = np.sum(np.abs(y_test - y_pred) / y_test.shape[0])
print("MSE=",error)
```



MSE= 0.454571665257975

Conclusion:

Applying Mini-Batch Gradient Descent to a supervised single-layer feedforward neural network balances the efficiency of batch processing with the robustness of stochastic updates. It enhances training stability and convergence speed for better model performance.

CO's Covered:

CO1. Implement basic neural network models.
CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.

Expt. No. 7

Title: *Implement a backpropagation algorithm to train a DNN with at least two hidden layers.*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 15/09/2025

Sign: _____

Experiment 7

Aim: Implement a backpropagation algorithm to train a Deep Neural Network with at least 2 hidden layers.

Software: Google Collab

Pre Lab-Questions:

Explain backpropagation algorithm for training deep neural network?

Backpropagation trains deep neural networks by propagating errors backward to update weights using gradient descent.

Post lab Questions:

Explain the role of hidden layers in back propagation?

Hidden layers in backpropagation extract intermediate features and propagate error gradients to update weights for learning complex patterns.

Theory: Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Working: Working of Backpropagation:
Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It Compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the bug report to get your desired output.

Backpropagation Algorithm:

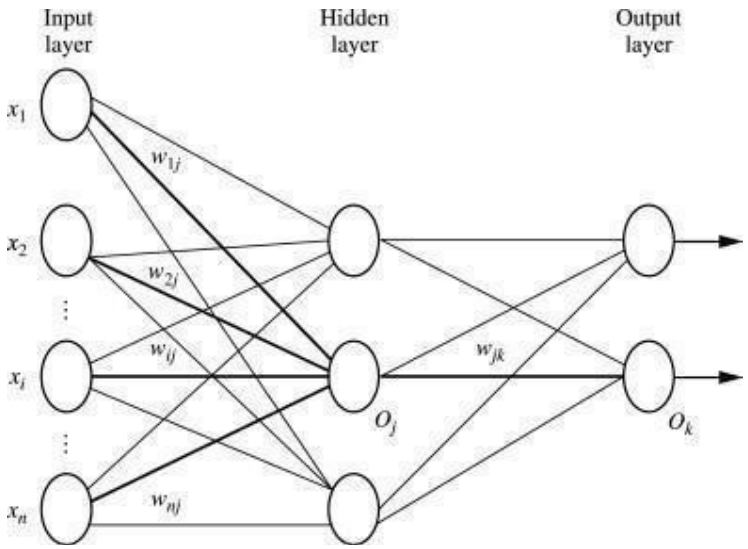
Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs Backpropagation Error= Actual Output – Desired Output **Step 5:** From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.



Training Algorithm:

Step 1: Initialize weight to small random values.

Step 2: While the step's stopping condition is to be false do step 3 to 10.

Step 3: For each training pair do step 4 to 9 (Feed-Forward).

Step 4: Each input unit receives the signal unit and transmits the signal x_i signal to all the units.

Step 5 : Each hidden unit Zj (z=1 to a) sums its weighted input signal to calculate its net input

$$z_{inj} = v_{0j} + \Sigma x_i v_{ij} \quad (i=1 \text{ to } n)$$

Applying activation function $z_j = f(z_{inj})$ and sends this signal's to all

Step 6: Each output unit y_k (k=1 to n) receives a target pattern corresponding to an input pattern then error is calculated as:

$$\delta_k = (t_k - y_k) + y_{ink}$$

Step 7: Each hidden unit Z_j (j=1 to a) sums its input from all units in the layer above δ_{inj}

$$= \Sigma \delta_j w_{jk}$$

The error information term is calculated as:

$$\delta_j = \delta_{inj} + z_{inj}$$

Updation of weight and bias:

Step 8: Each output unit y_k (k=1 to m) updates its bias and weight (j=1 to a). The weight correction term is given by:

$$\Delta w_{jk} = \alpha \delta_k z_j$$

and the bias correction term is given by $\Delta w_k = \alpha \delta_k$.

therefore $w_{jk(new)} = w_{jk(old)} + \Delta w_{jk}$

$$w_{0k(new)} = w_{ok(old)} + \Delta w_{ok}$$

for each hidden unit z_j (j=1 to a) update its bias and weights (i=0 to n) the weight connection term

$$\Delta v_{ij} = \alpha \delta_j x_i$$

and the bias connection on term

Step 9: Test the stopping condition. The stopping condition can be the minimization of error, number of epochs.

Program & Output

```
import numpy as np
X = np.array(([1, 9], [2, 5], [3, 6]), dtype=float)
y = np.array(([90], [85], [89]), dtype=float)

#maximum of X array longitudinally
X = X/np.amax(X,axis=0)
y = y/100

#Sigmoid Function
def sigmoid (x):
  return 1/(1 + np.exp(-x))

#Derivative of Sigmoid Function
def derivatives_sigmoid(x):
  return x * (1 - x)

#Variable initialization
epoch=2 #Setting training iterations
lr=0.1 #Setting learning rate
inputlayer_neurons = 2 #number of features in data set
hiddenlayer_neurons = 3 #number of hidden layers neurons
output_neurons = 1 #number of neurons at output layer

#weight and bias initialization
wh=np.random.uniform(size=(inputlayer_neurons,hiddenlayer_neurons))
bh=np.random.uniform(size=(1,hiddenlayer_neurons))
wout=np.random.uniform(size=(hiddenlayer_neurons,output_neurons))
bout=np.random.uniform(size=(1,output_neurons))

#draws a random range of numbers uniformly of dim x*y
for i in range(epoch):
#Forward Propogation
 hinp1=np.dot(X,wh)
 hinp=hinp1 + bh
 hlayer_act = sigmoid(hinp)
 outinp1=np.dot(hlayer_act,wout)
 outinp= outinp1+bout
 output = sigmoid(outinp)

#Backpropagation
EO = y-output
outgrad = derivatives_sigmoid(output)
d_output = EO * outgrad
```

```
EH = d_output.dot(wout.T)
hiddengrad = derivatives_sigmoid(hlayer_act)#how much hidden layer wts contributed to
error
d_hiddenlayer = EH * hiddengrad
wout += hlayer_act.T.dot(d_output) *lr # dotproduct of nextlayererror and currentlayerop
wh += X.T.dot(d_hiddenlayer) *lr

print ("----------Epoch-", i+1, "Starts----------")
print("Input: \n" + str(X))
print("Actual Output: \n" + str(y))
print("Predicted Output: \n" ,output)
print ("----------Epoch-", i+1, "Ends----------\n")
print("Input: \n" + str(X))
print("Actual Output: \n" + str(y))
print("Predicted Output: \n" ,output)
```

OUTPUT:

```
----------Epoch- 2 Starts----------
Input:
[[0.33333333 1.        ]
 [0.66666667 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.9 ]
 [0.85]
 [0.89]]
Predicted Output:
 [[0.84923164]
 [0.84921076]
 [0.85427789]]
----------Epoch- 2 Ends----------

Input:
[[0.33333333 1.        ]
 [0.66666667 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.9 ]
 [0.85]
 [0.89]]
Predicted Output:
 [[0.84923164]
 [0.84921076]
 [0.85427789]]
```

Conclusion:
x
Implementing the backpropagation algorithm to train a deep neural network with two hidden layers enables effective learning by adjusting weights based on error gradients. This demonstrates how deep architectures can model complex patterns through layered representations.

CO's Covered:

CO1. Implement basic neural network models.
CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.

Expt. No.8

Title: *Design and implement a CNN model for Digit Recognition*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 15/09/2025

Sign: _____

Experiment 8

Aim: Design and implement a CNN model for digit recognition

Software: Google Collab

Pre Lab Question:

Explain the CNN architecture?

CNN architecture consists of convolutional, pooling, and fully connected layers for automatic feature extraction and classification.

Post Lab Question:

Explain relation between input, output and filter size?

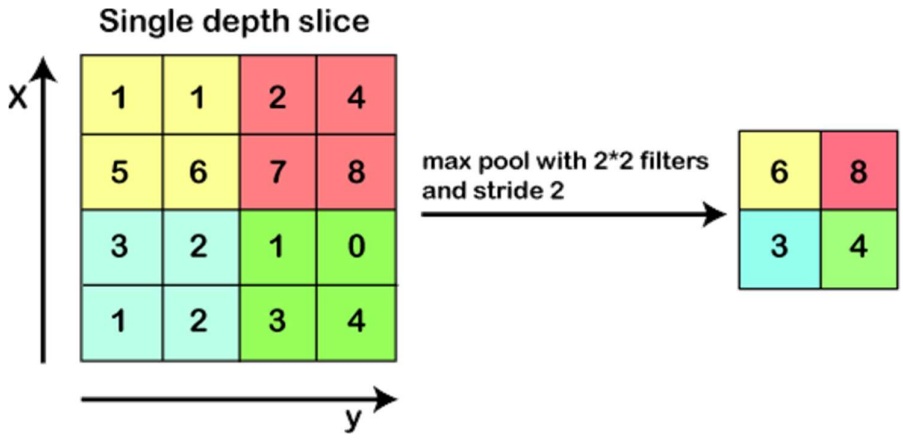
In CNNs, output size depends on input size, filter size, stride, and padding during convolution operations.

Theory: A convolutional neural network, or CNN, is a deep learning neural network sketched for processing structured arrays of data such as portrayals. CNN are very satisfactory at picking up on design in the input image, such as lines, gradients, circles, or even eyes and faces. This characteristic that makes convolutional neural network so robust for computer vision. CNN can run directly on a underdone image and do not need any pre-processing convolutional neural network is a feed forward neural network, seldom with up to 20.The strength of a convolutional neural network comes from a particular kind of layer called the convolutional layer. CNN contains many convolutional layers assembled on top of each other, each one competent of recognizing more sophisticated shapes.

CNN architecture

Generally, a Convolutional Neural Network has three layers, which are as follows;

- **Input:** If the image consists of 32 widths, 32 height encompassing three R, G, B channels, then it will hold the raw pixel([32x32x3]) values of an image.
- **Convolution:** It computes the output of those neurons, which are associated with input's local regions, such that each neuron will calculate a dot product in between weights and a small region to which they are actually linked to in the input volume. For example, if we choose to incorporate 12 filters, then it will result in a volume of [32x32x12].
- **ReLU Layer:** It is specially used to apply an activation function elementwise, like as max (0, x) thresholding at zero. It results in ([32x32x12]), which relates to an unchanged size of the volume.
- **Pooling:** This layer is used to perform a down sampling operation along the spatial dimensions (width, height) that results in [16x16x12] volume.


Single depth slice
X
y
1 1 2 4
5 6 7 8
3 2 1 0
1 2 3 4
max pool with 2*2 filters
and stride 2
6 8
3 4

Program & Output:

```
from keras.layers import Conv2D, Input, MaxPooling2D, Flatten, Dense
from keras.models import Sequential
from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

model = Sequential()

model.add(Input(shape=(28,28,1)))

model.add(Conv2D(32, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())

model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model.fit(X_train, y_train, epochs=10, batch_size=35, validation_data=(X_test, y_test))
```

```
Epoch 1/10
1715/1715 ━━━━ 12s 4ms/step - accuracy: 0.9002 - loss: 0.3207 - val_accuracy: 0.9856 -
val_loss: 0.0457

Epoch 2/10
1715/1715 ━━━━ 16s 4ms/step - accuracy: 0.9848 - loss: 0.0482 - val_accuracy: 0.9887 -
val_loss: 0.0357

Epoch 3/10
1715/1715 ━━━━ 10s 4ms/step - accuracy: 0.9902 - loss: 0.0307 - val_accuracy: 0.9883 -
val_loss: 0.0346

Epoch 4/10
1715/1715 ━━━━ 10s 3ms/step - accuracy: 0.9927 - loss: 0.0230 - val_accuracy: 0.9899 -
val_loss: 0.0295
```

Epoch 5/10
1715/1715 ——— 6s 3ms/step - accuracy: 0.9947 - loss: 0.0165 - val_accuracy: 0.9901 - val_loss: 0.0374

Epoch 6/10
1715/1715 ——— 10s 3ms/step - accuracy: 0.9955 - loss: 0.0133 - val_accuracy: 0.9916 - val_loss: 0.0306

Epoch 7/10
1715/1715 ——— 5s 3ms/step - accuracy: 0.9972 - loss: 0.0091 - val_accuracy: 0.9918 - val_loss: 0.0316

Epoch 8/10
1715/1715 ——— 10s 3ms/step - accuracy: 0.9970 - loss: 0.0089 - val_accuracy: 0.9909 - val_loss: 0.0330

Epoch 9/10
1715/1715 ——— 6s 3ms/step - accuracy: 0.9975 - loss: 0.0066 - val_accuracy: 0.9899 - val_loss: 0.0375

Epoch 10/10
1715/1715 ——— 6s 3ms/step - accuracy: 0.9979 - loss: 0.0061 - val_accuracy: 0.9902 - val_loss: 0.0425

```
import numpy as np
sample = np.expand_dims(X_test[7], axis=0)
# sample = np.expand_dims(sample, axis=-1)
sample.shape

np.argmax(model.predict(sample))
```

OUTPUT:

```
np.int64(9)

y_test[7]
np.uint8(9)
```

Conclusion:

Designing and implementing a CNN model for digit recognition enables automatic feature extraction and accurate classification of handwritten digits. This approach leverages convolutional layers to capture spatial patterns, significantly improving recognition performance over traditional methods.

CO's Covered:

CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.
CO3. Build and train deep learning models such as Autoencoders, CNNs, RNN, LSTM, GRU etc.

Expt. No. 9

Title: *Design the architecture and implement the autoencoder model for image compression*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 22/09/2025

Sign: _____

Experiment 9

Aim: Design the architecture and implement the autoencoder model for image compression.

Software: Google Collab

Pre Lab Question:

Explain the use of autoencoders?

Autoencoders are used for data compression, noise reduction, and feature learning by encoding and reconstructing inputs.

Post Lab Question:

Explain different types of autoencoders?

Types of autoencoders include vanilla, denoising, sparse, and variational, each with different encoding or regularization strategies.

Theory: Autoencoders are a deep learning model for transforming data from a high-dimensional space to lower-dimensional space. They work by encoding the data, whatever its size, to a 1-D vector. This vector can then be decoded to reconstruct the original data (in this case, an image). The more accurate the autoencoder, the closer the generated data is to the original.

Autoencoders are a deep neural network model that can take in data, propagate it through a number of layers to condense and understand its structure, and finally generate that data again. In this tutorial we'll consider how this works for image data in particular. To accomplish this task an autoencoder uses two different types of networks. The first is called an **encoder**, and the other is the **decoder**. The decoder is just a reflection of the layers inside the encoder. Let's clarify how this works.

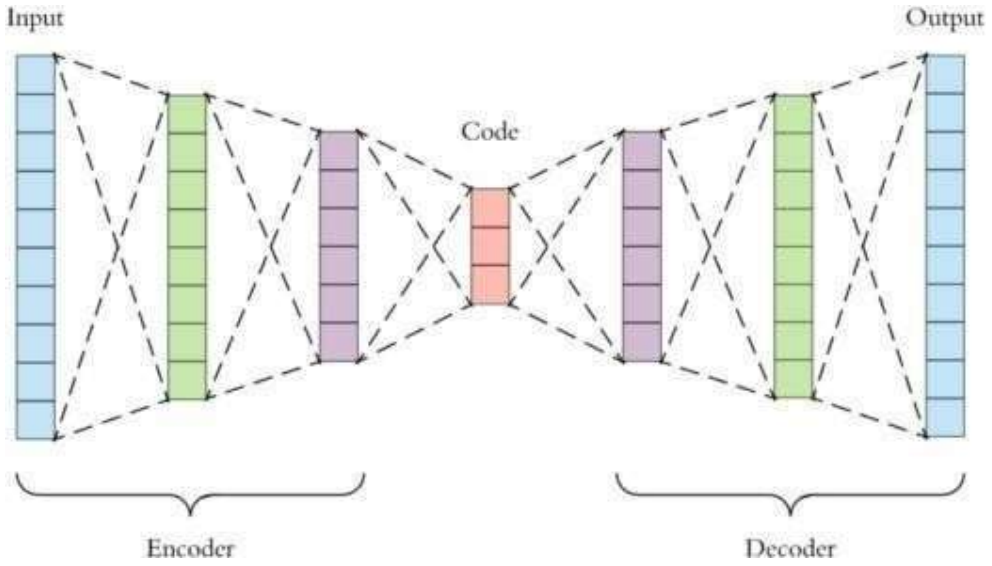
The job of the encoder is to accept the original data (e.g. an image) that could have two or more dimensions and generate a single 1-D vector that represents the entire image. The number of elements in the 1-D vector varies based on the task being solved. It could have 1 or more elements. The fewer elements in the vector, the more complexity in reproducing the original image accurately.

By representing the input image in a vector of relatively few elements, we actually compress the image.

Architecture of Autoencoders

An Autoencoder consist of three layers:

1. **Encoder**
2. **Code**
3. **Decoder**



- **Encoder:** This part of the network compresses the input into a **latent space representation**. The encoder layer **encodes** the input image as a compressed representation in a reduced dimension. The compressed image is the distorted version of the original image.
- **Code:** This part of the network represents the compressed input which is fed to the decoder.
- **Decoder:** This layer **decodes** the encoded image back to the original dimension. The decoded image is a lossy reconstruction of the original image and it is reconstructed from the latent space representation.

Program & Output:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
from tensorflow.keras.optimizers import Adam

# Load and preprocess data
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0

# Flatten images
x_train = x_train.reshape((len(x_train), 28 * 28))
x_test  = x_test.reshape((len(x_test), 28 * 28))

# Define encoding dimension
encoding_dim = 64  # Compress to 64 features

# Define input layer
input_img = Input(shape=(784,))

# Encoder
encoded = Dense(128, activation='relu')(input_img)
encoded = Dense(encoding_dim, activation='relu')(encoded)

# Decoder
decoded = Dense(128, activation='relu')(encoded)
decoded = Dense(784, activation='sigmoid')(decoded)

# Autoencoder model
autoencoder = Model(input_img, decoded)

# Compile the model
autoencoder.compile(optimizer=Adam(), loss='binary_crossentropy')

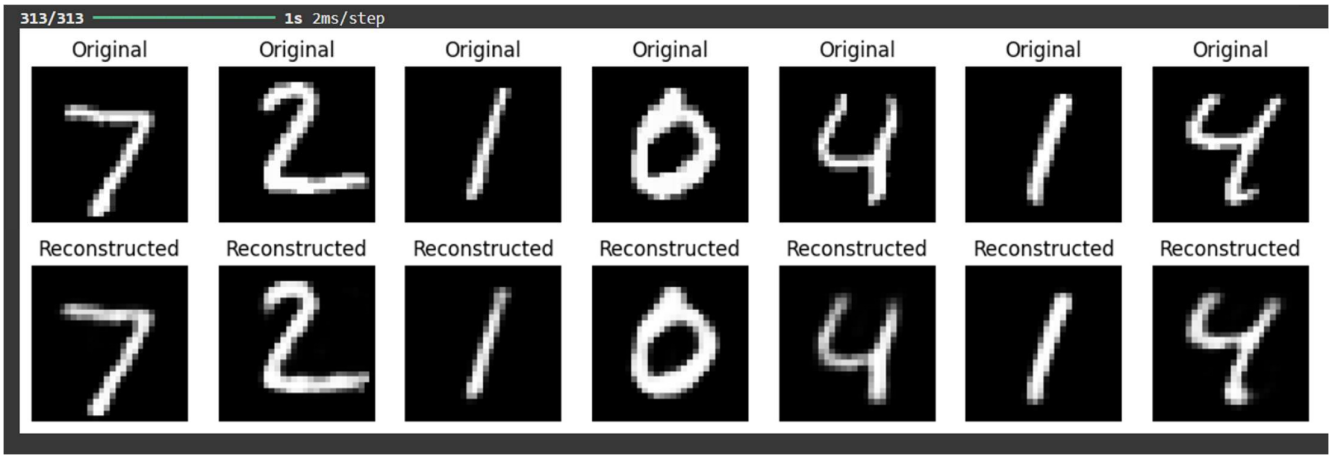
# Train the model
autoencoder.fit(x_train, x_train,
                epochs=20,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test))
```

```
# Encode and decode test images
decoded_imgs = autoencoder.predict(x_test)

# Display original and reconstructed images
n = 10  # Number of digits to display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Reconstructed
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis('off')
plt.show()
```

OUTPUT:



Conclusion:

Designing and implementing an autoencoder model for image compression enables efficient encoding of images into a lower-dimensional representation while preserving key features. This approach reduces storage needs and demonstrates how neural networks can learn compact, meaningful data representations through unsupervised learning.

CO's Covered:

CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.
CO3. Build and train deep learning models such as Autoencoders, CNNs, RNN, LSTM, GRU etc.

Expt. No. 10

Title: *Design and implement a LSTM model for sentiment analysis*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 22/09/2025

Sign: _____

Experiment 10

Aim: Design and implement LSTM model for sentiment analysis

Software: Google Collab

Pre Lab Question:

Explain the CNN architecture?

CNN architecture consists of convolutional, pooling, and fully connected layers for automatic feature extraction and classification.

Post Lab Question:

Explain relation between input, output and filter size?

In CNNs, output size depends on input size, filter size, stride, and padding during convolution operations.

Theory: The core idea of LSTM's are the cell state, and it's various gates. The cell state act as a transport highway that transfers relative information all the way down the sequence chain. You can think of it as the "memory" of the network. The cell state, in theory, can carry relevant information throughout the processing of the sequence. So even information from the earlier time steps can make it's way to later time steps, reducing the effects of short-term memory. As the cell state goes on its journey, information get's added or removed to the cell state via gates. The gates are different neural networks that decide which information is allowed on the cell state. The gates can learn what information is relevant to keep orforget during training.

Technically, LSTM inputs can only understand real numbers. A way to convert symbol to number is to assign a unique integer to each symbol based on frequency of occurrence. For example, there are 112 unique symbols in the text above. The function in Listing 2 builds a dictionary with the following entries ["," : 0] ["the" : 1], …, ["council" : 37],…,["spoke" : 111]. The reverse dictionary is also generated since it will be used in decoding the output of LSTM.LSTMs are primarily used in NLP and Time series forecasting. LSTM is really powerful that the results achieved using RNN's can also be achieved using LSTM.

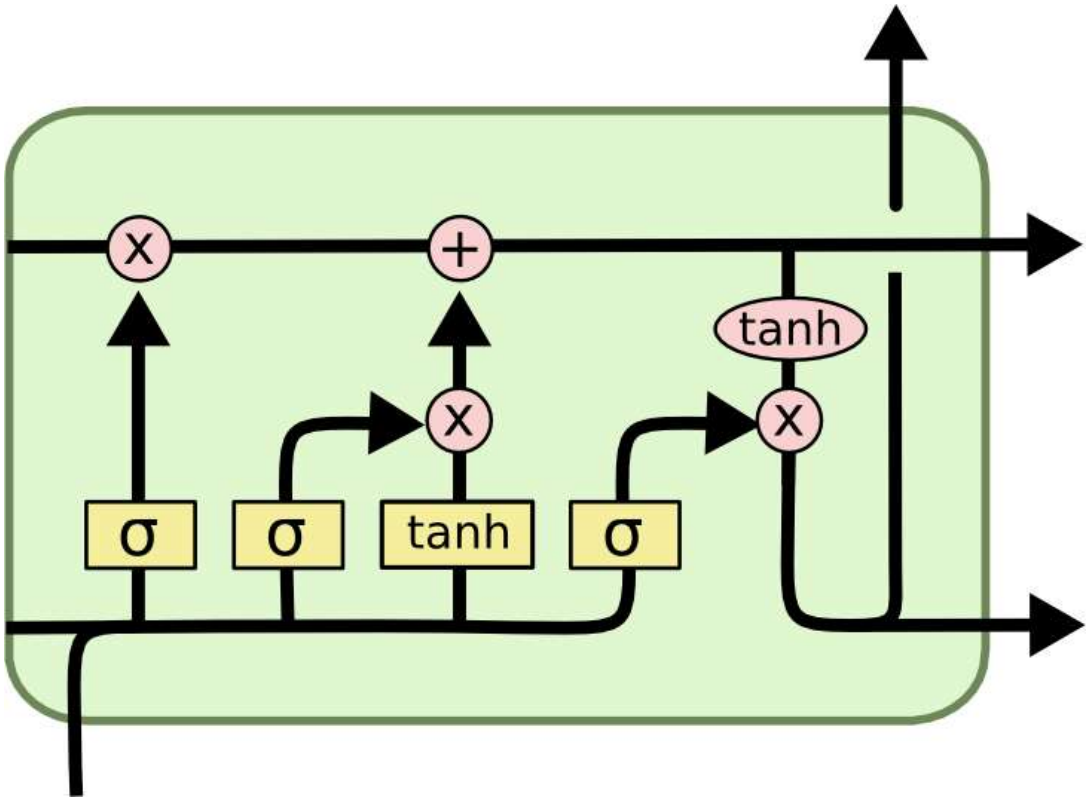
Sentiment analysis is the classification of emotions (positive, negative, and neutral) within data using text analysis techniques. Harnessing the power of deep learning, sentiment analysis mod els can be trained to understand text beyond simple definitions, read for context, sarcasm, etc., and understand the actual mood and feeling of the writer.

Architecture of LSTM

STMs deal with both Long Term Memory (LTM) and Short Term Memory (STM) and for making

thecalculations simple and effective it uses the concept of gates.

1. **Forget Gate:** LTM goes to forget gate and it forgets information that is not useful.
2. **Learn Gate:** Event (current input) and STM are combined together so that necessary information that we have recently learned from STM can be applied to the current input.
3. **Remember Gate:** LTM information that we haven't forget and STM and Event are combined together in Remember gate which works as updated LTM.
4. **Use Gate:** This gate also uses LTM, STM, and Event to predict the output of the current event which works as an updated STM.



Steps for performing Sentiment analysis using LSTM

Step 1) Importing the necessary libraries

Step 2) Read and train /test files using pandas

Step 3) Perform data pre-processing

Step 4) Import the API's from tensor flow to build LSTM model.

Step 5) Perform early stopping and save model checkpoints.

Step 6) Generate the graph function

Step 7) Define the prediction function

Step 8) Summarise the build model

Step 9) Predict the results after training the results.

Program & Output

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.datasets import imdb

# 1) Load IMDB dataset (already labeled)
vocab_size = 10000
maxlen = 100  # max number of words per review

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)

# 2) Pad sequences to ensure equal length
X_train_pad = pad_sequences(X_train, maxlen=maxlen, padding='post', truncating='post')
X_test_pad = pad_sequences(X_test, maxlen=maxlen, padding='post', truncating='post')

# 3) Build LSTM model
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=128, input_length=maxlen),
    LSTM(64, return_sequences=False),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()

# 4) Add EarlyStopping and Checkpoint
es = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
mc = ModelCheckpoint('best_imdb_lstm_model.h5', monitor='val_loss',
save_best_only=True)

# 5) Train the model
history = model.fit(
    X_train_pad, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2,
    callbacks=[es, mc]
)
```

```
# 6) Evaluate on test data
loss, acc = model.evaluate(X_test_pad, y_test)
print(f'Test Loss: {loss:.4f}, Test Accuracy: {acc:.4f}')


# Prediction

# 7) Word Index to convert raw text
word_index = imdb.get_word_index()

def encode_text(text):
    tokens = text.lower().split()
    encoded = [1]  # start token
    for word in tokens:
        encoded.append(word_index.get(word, 2))  # unknown token if not found
    return pad_sequences([encoded], maxlen=maxlen, padding='post', truncating='post')

def predict_sentiment(text):
    encoded = encode_text(text)
    prediction = model.predict(encoded)[0][0]
    return f"{'Positive' if prediction > 0.5 else 'Negative'} ({prediction:.2f})"

# 8) Try predictions
print(predict_sentiment("This movie was fantastic! I loved every moment of it."))
print(predict_sentiment("Worst movie ever. Don't waste your time."))
```

OUTPUT:

1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 29ms/step
Positive (0.85)
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 29ms/step
Positive (0.85)

Conclusion:

Designing and implementing an LSTM model for text data enables the capture of long-term dependencies and contextual information within sequences. This makes LSTM especially effective for complex natural language tasks like sentiment analysis and text classification.

CO's Covered:

CO3. Build and train deep learning models such as Autoencoders, CNNs, RNN, LSTM, GRU etc.

ASSIGNMENTS

Assignment. No. 1

Title:

Rubric	***Score (1 to 5)***
In time submission	
Presentation	
Total	

Performed On: _____

Sign:_____

Assignment. No. 2

Title:

Rubric	***Score (1 to 5)***
In time submission	
Presentation	
Total	

Performed On: _____

Sign:_____